

Project Week 07-08

March 26, 2025

0.0.1 2 Week 7 [Feb 10 - Feb 16]: Collaborative Filtering Recommender System

1. Based on the frequency of the most rated items computed in Week 6, implement the TopPop recommender system, which always recommends the same top-k items sorted decreasingly by the number of “high” ratings (e.g., 3) in the training split, train.tsv.

```
[40]: import pandas as pd

class TopPop:
    def __init__(self, k=10):
        self.k = k
        self.top_items = None

    def fit(self, file_path):
        # Read the training data (no header assumed)
        df = pd.read_csv(file_path, sep='\t')
        # Keep only the high ratings (e.g., >= 3)
        df_high = df[df['rating'] >= 3]
        # Compute the frequency of high ratings per item and sort in decreasing
        # order
        counts = df_high['item_id'].value_counts()
        self.top_items = list(counts.index)

    def recommend(self, user_id):
        # Return the top-k items (this model is user-agnostic)
        return self.top_items[:self.k]

top_pop = TopPop(k=10)
top_pop.fit('Data/clean_train.tsv')
```

2. Choose at least one neighborhood-based model and one latent factor model that uses the observed user-item ratings in the training set to predict the unobserved ratings. Report your choice of models.

```
[41]: from surprise import Dataset, Reader, KNNWithMeans, SVD
import pandas as pd

# Load your training data from the file (in 'Data/clean_train.tsv')
df = pd.read_csv('Data/clean_train.tsv', sep='\t')
```

```

reader = Reader(rating_scale=(1, 5))

# Prepare the dataset using the surprise framework
data = Dataset.load_from_df(df[['user_id', 'item_id', 'rating']], reader)
trainset = data.build_full_trainset()

```

3. Use 5-fold cross-validation on the training set to tune the hyperparameters of the chosen models (similarity measure and number of neighbors for the neighborhood-based model; number of latent factors and number of epochs for the latent factor model).

```

[42]: from surprise.model_selection import GridSearchCV
import random
import numpy as np

# Set the random seed for reproducibility
random.seed(42)
np.random.seed(42)

# Define parameter grid for KNNWithMeans (neighborhood-based model)
param_grid_knn = {
    'k': [5, 10, 20, 30, 40, 50, 70, 100],
    'verbose': [False],
    'sim_options': {
        'name': ['cosine', 'msd'],
        'user_based': [True] # We use user-based collaborative filtering
    },
    'random_state': [42]
}

# Define parameter grid for SVD (latent factor model)
param_grid_svd = {
    'n_factors': [3, 5, 10, 20, 50, 100],
    'n_epochs': [5, 10, 20, 50, 100],
    'random_state': [42]
}

# Perform 5-fold cross-validation for KNNWithMeans
grid_knn = GridSearchCV(KNNWithMeans, param_grid_knn, measures=['rmse', 'mae'],
    ↪cv=5)
grid_knn.fit(data)

# Perform 5-fold cross-validation for SVD
grid_svd = GridSearchCV(SVD, param_grid_svd, measures=['rmse', 'mae'], cv=5)
grid_svd.fit(data)

```

4. Choose an evaluation measure that is suitable for this task and justify your motivation in using it. Report the optimal hyperparameters together with the scores of your chosen measure, averaged over the 5 folds.

```
[43]: # We choose RMSE (Root Mean Squared Error) as the evaluation measure
```

```
print("\nKNNWithMeans Best Hyperparameters (based on RMSE):")
print(grid_knn.best_params['rmse'])
print("Average RMSE for KNNWithMeans:", grid_knn.best_score['rmse'])

print("\nSVD Best Hyperparameters (based on RMSE):")
print(grid_svd.best_params['rmse'])
print("Average RMSE for SVD:", grid_svd.best_score['rmse'])
```

```
KNNWithMeans Best Hyperparameters (based on RMSE):
{'k': 70, 'verbose': False, 'sim_options': {'name': 'cosine', 'user_based':
True}, 'random_state': 42}
Average RMSE for KNNWithMeans: 0.9100087207501553
```

```
SVD Best Hyperparameters (based on RMSE):
{'n_factors': 5, 'n_epochs': 20, 'random_state': 42}
Average RMSE for SVD: 0.8440025192355
```

5. Run the models with the optimal hyperparameters to the whole training set.

```
[44]: # Re-train the models on the full training set using the optimal
      ↪ hyperparameters from the grid searches
```

```
# For KNNWithMeans
knn_best = grid_knn.best_estimator['rmse']
knn_best.fit(trainset)

# For SVD
svd_best = grid_svd.best_estimator['rmse']
svd_best.fit(trainset)

print("KNNWithMeans model trained with optimal hyperparameters.")
print("SVD model trained with optimal hyperparameters.")
```

```
KNNWithMeans model trained with optimal hyperparameters.
SVD model trained with optimal hyperparameters.
```

6. Use the final models to rank the non-rated items for each user. This ranking will be used for the evaluation part next week.

```
[45]: from collections import defaultdict

# Build the anti-testset (all user-item pairs not in the training set)
anti_testset = trainset.build_anti_testset()

# Get predictions on the anti-testset using the KNNWithMeans model
predictions = knn_best.test(anti_testset)
```

```

# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted
# rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:
    x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using KNNWithMeans:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")

# Get predictions on the anti-testset using the SVD model
predictions = svd_best.test(anti_testset)

# Group predictions by user
user_recs = defaultdict(list)
for pred in predictions:
    user_recs[pred.uid].append((pred.iid, pred.est))

# For each user, sort the recommendations in descending order of predicted
# rating
for user in user_recs:
    user_recs[user] = [iid for iid, _ in sorted(user_recs[user], key=lambda x:
    x[1], reverse=True)]

# Display sample recommendations for a few users
print("Sample recommendations using SVD:")
for user, recs in list(user_recs.items())[:5]:
    print(f"User {user}: {recs[:10]}")

```

Sample recommendations using KNNWithMeans:

User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0C67HCGBR', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',
 'B00H35YIJE', 'B079TLFL33', 'B000T9L7W2', 'B007MY5BDI', 'B00H4PEMM6',
 'B09YDBKT7M', 'B0002D0Q2W']

User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B0C67HCGBR', 'B0B8M5FJB6', 'B000T9L7W2',
 'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKTYTPQ', 'B001W99HE8',
 'B000J5UEGQ', 'B01DBS2U9G']

User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B005M0MUQK', 'B0BRS6V8G4', 'B078L11275',
 'B09R6KV6QX', 'B07YK57N2M', 'B00RX5HQS4', 'B0BQ4HSC9', 'B086QM1F75',
 'B07R3S93K8', 'B09WF82F1V']

User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B06XB3FQKB', 'B0BPJ4Q6FJ', 'B0B8M5FJB6',

```
'B079TLFL33', 'B007MY5BDI', 'B00H4PEMM6', 'B09YDBKT7M', 'B0BXT384GR',
'B079P9LDHN', 'B0BPKH4HB2']
User AESDNHQRRZEWTF7A7TFFFTSNY5Q: ['B0BGQZNQ53', 'B07L5B64RG', 'B07Y94MSG9',
'B0CB98SMQR', 'B079NS31NK', 'B00CIHB8FY', 'B000SHQ1QC', 'B0BQ4HSC9',
'B00IZA1GI2', 'B000U0DU34']
Sample recommendations using SVD:
User AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: ['B0BPJ4Q6FJ', 'B00H35YIJE', 'B079TLFL33',
'B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M', 'B079P9LDHN', 'B0BPKH4HB2',
'B07JCW1KGG', 'B001W99HE8']
User AG44UYFDZHI6FRBQSLDWOGIEOY4A: ['B000T9L7W2', 'B007MY5BDI', 'B09YDBKT7M',
'B079P9LDHN', 'B001W99HE8', 'B01DBS2U9G', 'B07CBT4SYD', 'B09M7DPHCS',
'B08DM9NFLH', 'B0BTC9YJ2W']
User AE7P3HIBI3UDLCJLUPUZQWYVPEWA: ['B0BT2ZRCY2', 'B07DK59QNR', 'B000U0DU34',
'B09G5KLKX2', 'B01DE4D4LU', 'B0B8F6LD9F', 'B0BT9R8MMV', 'B07N5MJDBF',
'B07S19XSPV', 'B0CB98SMQR']
User AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: ['B07N5MJDBF', 'B0BT9R8MMV', 'B0BT2W3TTM',
'B07N2HQ1T7', 'B000U0DU34', 'B09M7BF885', 'B09G5KLKX2', 'B0BRS6V8G4',
'B01DE4D4LU', 'B07DK59QNR']
User AESDNHQRRZEWTF7A7TFFFTSNY5Q: ['B000U0DU34', 'B0BT2ZRCY2', 'B0BT9R8MMV',
'B0B8F6LD9F', 'B01DE4D4LU', 'B09G5KLKX2', 'B07C9YCY5J', 'B0BQ4HSC9',
'B07N5MJDBF', 'B0CB98SMQR']
```

0.0.2 3 Week 8 [Feb 17 - Feb 23]: Evaluation of Recommender Systems

In this session, we will discuss how to evaluate a recommender system. Specifically, let us evaluate all your recommender models from Week 7 on the preprocessed test data split studied in Week 6. You need to: - Measure the error of the system's predicted likelihood of rating for the items (Root Mean Square Error, RMSE). - Discuss the limitations of this metric.

```
[46]: from surprise import accuracy

# Load the preprocessed test data (assuming same structure as train data:
# user_id, item_id, rating)
test_df = pd.read_csv('Data/clean_test.tsv', sep='\t')

# Convert the test DataFrame to a list of (user, item, rating) tuples
testset = [tuple(x) for x in test_df[['user_id', 'item_id', 'rating']].values]

# Evaluate SVD model on the test set
svd_predictions = svd_best.test(testset)
print("SVD RMSE on test set:")
accuracy.rmse(svd_predictions, verbose=True)

# Evaluate KNNWithMeans model on the test set
knn_predictions = knn_best.test(testset)
print("KNNWithMeans RMSE on test set:")
accuracy.rmse(knn_predictions, verbose=True)
```

SVD RMSE on test set:
RMSE: 0.9795
KNNWithMeans RMSE on test set:
RMSE: 1.0619

[46]: 1.0618532415444242

Now, we are interested not in whether the system properly predicts the rating of these items, but rather whether the system gives the best recommendations for each user. To evaluate this, generate the top-k (with $k = 10$) recommendation for each test user. Based on the top-k recommendation list generated for each user, and using the test data split5, compute: - Hit rate, averaged across users. - Precision@k, averaged across users - Mean Average Precision (MAP@k) - Mean Reciprocal Rank (MRR@k) - Coverage

Discuss the advantages and disadvantages of these metrics. Compare the two types of CF recommender systems and the TopPop system that we have defined so far. Which one works best? Why? What are the advantages and limitations of each approach?

```
[47]: # Prepare ground truth for each test user (using binary labels)
# Relevant (1) if rating >= 4; not relevant (0) otherwise.
ground_truth = defaultdict(set)
for _, row in test_df.iterrows():
    if row['rating'] >= 4:
        ground_truth[row['user_id']].add(row['item_id'])

# Function to generate top-k recommendations using a given model.
# For personalized models we use the anti-testset from the training set.
def get_top_k(model, trainset, k=10):
    # Build the anti-testset (all unseen items for every user in the trainset)
    anti_testset = trainset.build_anti_testset()
    predictions = model.test(anti_testset)

    recommendations = defaultdict(list)
    for pred in predictions:
        recommendations[pred.uid].append((pred.iid, pred.est))

    # For each user, sort predicted items in descending order and select top-k.
    for user in recommendations:
        recommendations[user] = [iid for iid, _ in
    ↪sorted(recommendations[user], key=lambda x: x[1], reverse=True)[:k]]
    return recommendations

# For TopPop (non-personalized), the top-k recommendations are the same for
↪every user.
def get_top_k_for_top_pop(top_pop, test_users, k=10):
    recs = {}
    top_items = top_pop.top_items[:k]
    for user in test_users:
```

```

        recs[user] = top_items
    return recs

# Generate recommendations for each model.
# Retrieve the list of test users from ground_truth.
test_users = list(ground_truth.keys())

svd_recs = get_top_k(svd_best, trainset, k=10)
knn_recs = get_top_k(knn_best, trainset, k=10)
top_pop_recs = get_top_k_for_top_pop(top_pop, test_users, k=10)

# Get the total catalog size (we use all items from the training split)
total_items = trainset.n_items

# Evaluation function for hit rate, precision@k, MAP@k, MRR@k and coverage.
def evaluate(recs, ground_truth, k=10, total_items=total_items):
    hit_rates = []
    precisions = []
    avg_precisions = []
    reciprocal_ranks = []
    rec_items = set()

    for user, true_items in ground_truth.items():
        recommended = recs.get(user, [])
        rec_items.update(recommended)
        # Hit rate: at least one relevant item recommended.
        hit = 1 if set(recommended) & true_items else 0
        hit_rates.append(hit)
        # Precision@k: fraction of recommended items that are relevant.
        precision = len(set(recommended) & true_items) / k
        precisions.append(precision)
        # Average Precision (AP@k)
        ap = 0
        hit_count = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                hit_count += 1
                ap += hit_count / (i + 1)
        if len(true_items) > 0:
            ap /= len(true_items)
        avg_precisions.append(ap)
        # Reciprocal Rank (MRR@k): rank of the first relevant recommendation.
        rr = 0
        for i, item in enumerate(recommended):
            if item in true_items:
                rr = 1 / (i + 1)
                break

```

```

        reciprocal_ranks.append(rr)

    hit_rate = np.mean(hit_rates)
    precision_at_k = np.mean(precisions)
    map_at_k = np.mean(avg_precisions)
    mrr_at_k = np.mean(reciprocal_ranks)
    # Coverage: unique items recommended over catalog total.
    coverage = len(rec_items) / total_items

    return hit_rate, precision_at_k, map_at_k, mrr_at_k, coverage

# Evaluate each recommender.
svd_metrics = evaluate(svd_recs, ground_truth, k=10)
knn_metrics = evaluate(knn_recs, ground_truth, k=10)
top_pop_metrics = evaluate(top_pop_recs, ground_truth, k=10)

print("SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", svd_metrics)
print("KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", knn_metrics)
print("TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =",
      ↪top_pop_metrics)

```

```

SVD:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.0825242718446602,
0.00825242718446602, 0.00610439651075204, 0.02219140083217753,
0.11787819253438114)
KNN:    Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.10922330097087378,
0.011650485436893206, 0.00841981197641613, 0.03142144398212359,
0.587426326129666)
TopPop: Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.25728155339805825,
0.032524271844660196, 0.03386359713699836, 0.11868354137771613,
0.019646365422396856)

```

Analysis on the effect of the long-tail Sort the users by number of ratings in the train set and select the top 20% and last 20% of users.

```

[48]: # Compute the number of ratings per user from the trainset
user_rating_counts = {trainset.to_raw_uid(uid): len(ratings) for uid, ratings
    ↪in trainset.ur.items()}

# Sort users by their number of ratings
sorted_users = sorted(user_rating_counts.items(), key=lambda x: x[1])
num_users = len(sorted_users)

# Determine the 20% count for top and bottom users
num_20_percent = int(num_users * 0.2)

# Select bottom 20% (users with fewest ratings) and top 20% (users with most
    ↪ratings)

```



```

bottom_users = [user for user, count in sorted_users[:num_20_percent]]
top_users = [user for user, count in sorted_users[-num_20_percent:]]

print("Total number of users:", num_users)
print("Bottom 20% users and their review counts:")
for user in bottom_users:
    print(f"{user}: {user_rating_counts[user]}")
print("Top 20% users and their review counts:")
for user in top_users:
    print(f"{user}: {user_rating_counts[user]}")

```

Total number of users: 800

Bottom 20% users and their review counts:

```

AF2MGAQBAT3E4XQC7NNAQFYG4MIQ: 1
AGMYRILPTJDOK7WL50YHYGRYVXM: 1
AEMZMINSQYQBEDD3TPDQZZFFWIQA: 1
AGDEYWGZKNGR36MR07ZT3AB45AXA: 1
AHMUAUOHLITDKIKI4C3KVRWIBMAQ: 1
AFFOT7AORQAWKU6KKPDHR3FOA5AA: 1
AEXXX2MQRH3AHZMBZBP4RDNGRRWQ: 1
AFGJ42MV2H2CHEZNN3JPQ3RSYUHQ: 1
AESA6T5VF3ADXH5GYRX2RDMN6I6Q: 1
AEWWDVYR4YKLL0ILS55VQISDPGYQ: 2
AHOUKQ3IRAEAMEHDB6U4AMB366NQ: 2
AGSP5XAQPQBUXZHEZSC65FD7NOQ: 2
AH2RITMRQC3JL44WADB4GV23AFVQ: 2
AF63VW7YLF42A6STVGBLACD7RUWA: 2
AHMMZCKZMV7U5SPYVE6IG2YACR7Q: 2
AHMVWBJ4FNFOXJ6BXA5M7CEDHSNA: 2
AHRPODCFJ7UNWQKCLLBKN6BQIN7A: 2
AHWAYCEX3NCWJD32JTFVBFXKHUHQ: 2
AGHARSG3LRWDNOAM3HVDWKJJMS3Q: 2
AE2NWSTL7JJWOCBKZCF6KDQIZQ: 3
AGRGLPIRFOMTUIBIYJOUTTGZHABA: 3
AETS7HNSQIFRSMRCWQMHIWZ2IBOQ: 3
AGHJMMZCZKGM2XC7E32JDSAGRWHQ: 3
AHUIUCGSU06B2KRQLQ2UWRYXTH5Q: 3
AF72XXUN4AGVL4K45BIXLKDIZRTQ: 3
AEYWZ24XFZCRP3PXNJB2TLXFX47A: 3
AEWQYNFY055WCN2IVUVQET30FS6A: 4
AE6ZIS54JBDY3UPZQGSXRJMPQ30A: 4
AH44K3RPIQMVGW2G4EB5FPVIVUEQ: 4
AGYK2YYS35CDU2Z77ZQR7L3IAX4Q: 4
AFPBNLIVMV6PSXJFUYLMO3EYWBPA: 4
AGNQMZSUPRA3BEM44ITFIFXKGPXQ: 4
AH2APCDO2UOVZXSGM7VZR3XMWR3A: 4
AFVHSDXVXIX3ZM6JB32LZEAYTFTQ: 4
AEGPH37EQW2VFMQTLTMTD26JH6ZQ: 4

```

AEHDYSX54B3NYUIIEIHEGHIGJNMQ: 4
AF4VYLI2QY2N7S65KZHMIET70GOQ: 4
AG6FYGUMX04TSTPFWPFTNHJXC7WQ: 4
AGBQ3TX4QYWQQOW2DBADLGOJHQGA: 4
AGB5CRFACFQG6C30B2TSE5JC5FHA: 5
AH6QSJEXBISHMQ3M3THWZAKMB72A: 5
AFPU3TXAOZ2VSTJAV5SDRF2XA6CA: 5
AFLMX52QL4FWD3PYMYQWWE36QHMA: 5
AEXLRZ34GTMM35NQRDZ4CDKKXLIA: 5
AF3FYICX6MT2J3E7RBI6WGC2DSVA: 5
AEOG5GP4YYOR6T7H4DTLTH5VXEVA: 5
AGAF5MHBOW2UYHNHSBXIXLKJXIKQ: 5
AGBVX6ERCZ5FBYQCLQNGTWJWKC5A: 5
AF2GPIP26H7E5EHDWC6P7RKZ30UQ: 5
AFIMYZ3UMWWZKR2ZFTOQZ2BDGA6Q: 5
AHLRDVX6ROCCPNXTCZLY2KPVLCHA: 5
AFOCHHI44XKMG44TIDXRHY7R2H2Q: 5
AHICAEXRGOBNEKCK2KQYUI44SMVQ: 5
AE23LDQTB7L76AP6E6WPBFVYL5DA: 5
AE6JDIRVPTKC32Y7X3QQ3WAY2LIQ: 5
AGFSYLEFUQAJDTR55TENS6BZ56XQ: 5
AFJJWM45JXP7ONUSAYQWWEIVDLQ: 5
AFDZU5HCDZTKTC6LL2KI5DQH2IA: 5
AEQJ4C6IRPWITBGZ3PVJJB6DLW3Q: 5
AHTOQ6BJJR67CXFLW7CGRTLA3A3A: 5
AEKTSZJEHS2AFYBWFIMASB43ZC5A: 5
AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: 6
AHSEAF6NAXXWPPY3CSL2K24G5IOA: 6
AHL4BCH2033HFN7KSBTIJUHCJYDA: 6
AHG4XUNNKMZUSMC7UI3YRS5YMD7A: 6
AFRUTEYFUJDAUASSQADZ3UQJHXAA: 6
AHETQRJ33CMSEUL4SDS6CJMHDU6A: 6
AHGODYYYGTJF05RXGWPSKFVXS25PQ: 6
AFHIW6ZJRTCQCT2BKFRDPY2RQWLA: 6
AENF7IROS6474V6TOKAONGVZHMVQ: 6
AG34YJOTCJWCEA6XGG3NWWHDSQIQ: 6
AG5CLB4M2P4GCGANQS4WS3JGHFOQ: 6
AGPQVGNBINLAEQ6IWHJBGSL6YXJA: 6
AGMIECBRRAS3YUEBQNL7M766ZVQ: 6
AHAZ5ZPYUOSJP6GUP5AM2VMUFNTA: 6
AHEIWAU3PKILH7VPQZGXPBS5BKEQ: 6
AEUZQ2IHGQE75ZWT6SQ6NQTWPKPKQ: 6
AGSVLMHAE6DKL34BW5AJANZ2HP3A: 6
AG4V6SXKSVY05TTRJQRPLBSK25HQ: 6
AGRWNQCKOVXJEAHMQ5RWUMGVAP5Q: 6
AENFQD5EQBOPKB6TB7A0AD5QQCOA: 6
AG4VWLJ2X7PIBYDXYYUNFY5ONDRQ: 7
AG5S2NUNLTCXHX7H5UYE5LZXJK7A: 7

AEWLK6HFFQ44AJIC4YHECUU7L4QA: 7
AG6MB6ECQOKI5LMTUXYE7NCQYQQA: 7
AGOZJU464U2DQK5MI4YNE4CSCLCQ: 7
AEH4Y332KQ4FVFT5VTSILR2RGJ4A: 7
AGBWWY2Q4KM2IH3WA45BSPOCKRJ0A: 7
AEQHYWT6W3GRY36WBNLB7J6HWHPA: 7
AHLU6AWIBESPT72U26FRE2PVI5JQ: 7
AF204GMP7Z6ZV7WZGJM5MGT27GAA: 7
AG77NH35B3YCLZH VX7ZBWGCPKU7Q: 7
AHUZL5QVP2SC6G0XSKGNAYUZQZ6A: 7
AGAZUH6XYIKXLJHGMSB5WA5ZV2KQ: 7
AFCH3BNUJBRNZP53DEJTCOZ26BIA: 7
AEMS FZ4NLED647BP3ZCQEY3FNGKA: 7
AHVGICSVXM66E4SXX23PH7R3Q74Q: 7
AH7I57UVCR75DY405MDVVGWT2CJQ: 7
AE4LGWEQHG3GNC3YEIWKBSZN34Q: 7
AHYQKMLIXHKFDHC7K2Y4JXM4YLOQ: 7
AFTF5Z4H3CXBKB2HET3J72K6MTUQ: 7
AEMISR DYQCHSNOXKV6TQ67X2TIOA: 7
AFR6WKFBTKBG7HH7MOIHP7Y4IJQ: 7
AETRLZMHXJWHRPFLNPAT5KWVN2UQ: 7
AEOUOKA2NPLBOSG73KGJTVQIOK6Q: 7
AGSVU7HWV6AHG4QFFTVWDXNM3UJA: 7
AFN4MT6SZPZWDQ5D5AGJ6WP3E3YQ: 8
AEYU40X60E3CJ6BQH2KLSNXPKRPA: 8
AHMKNGTBHAIBOSCM03N7FDGPTIA: 8
AFKIK7F3PKMBCI2IKCQWH6ZY5DEA: 8
AGASE3XTUSEWWXX6RV5KD4NDXFAA: 8
AFIEQ7BNYXBQTLYIJRTBNKWQTKEQ: 8
AFDS5HV3F3ONORE7U7PMXMVMZAU: 8
AFNKANPQUS5G3ENXFLAJ4CHWQGFQ: 8
AGY5HQSNTLTDJRG533ZDFUAHMRQ: 8
AHC23DBZJ2VVCK6UGCM7NPFP7LQA: 8
AE76MPFI6EGHS7HMMTW3SXCLVKA: 8
AER5I36DVG SOMSGX3ZQHYZGPESFA: 8
AEVUPHNKTJYUVF3LVHMYVZT4WLUQ: 8
AFUD46FQ5QP2KHEPMCNSWXMDZ5MA: 8
AFG7G0J4XZ6MQA7EXJQ6BCK3Y2YQ: 8
AEJH4HUMFAN62AII4PTCGRGGA7Q: 8
AFVSSJ5QSQBZBTAVR5XHF6UQQVBA: 8
AHPIKIUV07ZBNW4K40LRCDRWLQYA: 8
AF6NUCZR3MMBLJJCDNZS6PKZTOYQ: 8
AGTQ4C5UOLPC6RNAX57302TU5S3Q: 8
AFRFYELVMFERMILMAIN3NY7YOPSA: 8
AESL6WMA3BO3ZXPTK6EAQJIOFTUQ: 8
AE5D5Q2I4H7CLGGXKPQPMXOG6JLA: 8
AGFIGINQSL73HWAKFPGSIYO3KU5Q: 8
AFJ66RVSHHQWD3MZRM6JNQB5NGXA: 8

AHCLEFILWZQNDMN4L3LC3DJLZNSQ: 8
 AGWKQRZOH2KLRB3W72ZFRMOTZRQ: 8
 AGR2HGNGBX6TFVFHQFKXRPP4B22Q: 8
 AHBV7TH4Z3WRRKXUIDAOPVUBIR4Q: 8
 AHHJZCFUMYNQ4M2VAYCDX537ZBRQ: 8
 AE4ZMURD4I73BSFCFQPGOWBOHXIA: 8
 AG5QJWMXSGR6QK7CTCJXRD70SOQ: 8
 AF5QL2TW5Z63NYQ7IKBJVHB5T2CQ: 8
 AF3JMCAJ6IICLZJRLDYWCPSEN2A: 8
 AHFWBUOIMC5ESSI6SQWTVXDYQ55Q: 8
 AHFEX4TZQJHG6BTYRZQED45HVT6A: 8
 AHNMR3M7H4JDTE05LXIKZC4DJUUA: 8
 AEFPGS00IM5GMCXPX7FOCCH3UF6ZA: 8
 AF3ZYMHEW4XP4NHUIZXKVVSAXFQ: 8
 AE7P3HIBI3UDLCJLUPUZQWYVPEWA: 9
 AFVLYQWBCIHXLH6X2FSXRMAHXA: 9
 AEHGPRL2ZLF6TY6JWBNFJNIDTE6Q: 9
 AHSEUPHWABAKNUWPZJOLLGDM47FA: 9
 AG2OYBLXDT4PNZZDRKYHEELJ75YQ: 9
 AFXVMXIAMROWJSIZ2IQBTX2GF4LA: 9
 AGLSNCKCMBZTZKPFN3BY2ANTARQ: 9
 AEYIGWWXTT5AD4ZONPSHMK53WR2Q: 9
 AGSWTRJPZ7AVS2C33JGND5BQ6SMQ: 9
 AHVFEBH4PD2V3XK2AZ44LFXTPZUA: 9
 AGL72EW24IMR04K5EJ6S3QGG365Q: 9
 AG6CZXXQXXL3L7TE7XVHTW3DYKYA: 9
 AFV2YRAITESEBOVQYURW6VGM24MA: 9
 AH22LHGRVRIAJI32SBDMHNOKPPMQ: 9
 AFHER2NWLJQHNQ4BRB4NZAG46VCQ: 9
 Top 20% users and their review counts:
 AGVJNAG7UKKZH2XTBDZ2HD5DOJCA: 15
 AENBPEIFKODS2DZB7JNJMYQTZ46A: 15
 AFZ4XEURL7VMVPEKG5SHU30AULQ: 15
 AFTN36CZUYIXDWJYJD702LBXYOEA: 15
 AEH60LQHQQVSIU7BDLIHSMIBOGNQ: 15
 AHI4QESUN4OC3NMOB6M5KCE6WBDQ: 15
 AEEDYZE7SWKKVMVWYZD35VYCMWQ: 15
 AFLTPN2GUQICRVFNIGT7X6QUW5JQ: 15
 AG6QMKE26TOI4SJCPJD2F3APWE7Q: 15
 AFVN3Z7WL2CLXIBUTHYR6K6Y7YJA: 15
 AHRDQWA7SWNXDYHRPEZ63D5Q7B5Q: 15
 AHPMJNUYUX4QOZCKLN5R34BB5EQQ: 15
 AEND7BD36I5IUIVPDZJAQXBLR5CA: 15
 AG3TBH03TDHQXXZR6IUHEZGAZ2MQ: 15
 AENNORCDOTT2KFT5WUIK7PJ2EXO: 15
 AHOZBVGYYQXJE4NJRSPWY52ILAKQ: 16
 AFGQTS5VSAM326L06ZKAIWIFK5A: 16
 AEEWF5YTHNKKMXQH542QEZB07RGQ: 16

AFDHTBSFETGIVMU733ZGGTY63EEA: 16
AE03CZ4KPIA3I47EADFS43KVFX2A: 16
AGZEDKJEUXQ3KK5D462HQZYTQWXA: 16
AFYONFWGXVIWRVF2SHUIPM6F2C3A: 16
AGBCLRMS70W3N3T44G06W6M6RNBA: 16
AFSUK4CHQFQYHUISLHWFDI3T7EGA: 16
AHICFG7RTZ5XSIGBUFK5XVOLZIJQ: 16
AHA4V44CE4KWD2NFAN7RAB4HJJZA: 16
AGJAAVMYOSWYRC3HRWGAWHJFS4WA: 16
AEESX3HNU6500FXP7VYMTQG50ZLA: 16
AHHTZKWL3I3MRXLE2RD5T320A5Q: 16
AE42Q4LRELZEBXTDYI6PYTWZEOZQ: 16
AELNRQWRJQYTAKSHAXR3LBUGIFGA: 16
AG7RRPB7KFXRKFIUEFLCLZ2KU23Q: 16
AF5KJIMQXNEY7M5U5YM43QPC04PA: 16
AGJX6XVM4R33WWI4R4PNSXVFZ7RA: 16
AGWAUKTSERZ304NYZWZS70K5ABPQ: 16
AFVX0XPDFHFXLCYM6H2HM5PUOCMQ: 16
AGU6C6L32G5WTF334TTCEWIGGVA: 16
AGTGRAU2XMEYIX4WPQPNY3JGAUNQ: 16
AFWITPOXSUXLPPPZTIHDHV7QZIZQ: 16
AEJVVNXW55ISA30ATO64QX5WXYQ: 16
AFAZC54MQKYVDQH7NY4GJ7BDBCMQ: 16
AGNZ55VEXGISW030H7F4CTP6HNRA: 16
AGIBZ4QTKA34IT3EVMH52FKJERQ: 16
AEF2RX7ZF5KKRXYURVRY6AKZGLSQ: 16
AEBGYELMA4BKQPPWGIREPQXDLGQA: 16
AEBD7F5ITO4VSB4V7L2G35YBSQ: 16
AFB60B3NPQ3ZCIFC7CQG6JL6J7DQ: 16
AEIGNQD2PURKBD57LI4CYPUF2SNA: 16
AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 17
AFGMXXQWDSRK4H7UWEFZKF603JOQ: 17
AH3UBUKNUPJRPZ4MFG4JFE2BD2RA: 17
AHWXEOF3AV66R44YXMV60L3GTLWQ: 17
AGDR5UXGHUY2HYK2760UE7J4T73Q: 17
AG23AW56WZ7KPYYAS4A3YUDTYWCQ: 17
AFEPLHWMMPUHJCXBCDWHIDLE2A6Q: 17
AHXZMBZM7TCTGR6YQ7MI7LEAZ7QA: 17
AE37RAW77LNOTEDKMDKGXSGQHD5Q: 17
AHZER36Y6IL7USHGBCELVMN56VOQ: 17
AGY53AAWLQK5077H2YKXCURQHNTQ: 17
AFJAAQX3CJRKW3ETBKBU4XTHMQKA: 17
AGFCZHTATBUI4HMETZZ5DXULTZJQ: 17
AGOLVETLIB3P4APNHI1KQXZB4Y5Q: 17
AGMTCMP27WCMXZ3R3FJ5F4KDABA: 17
AG43KQOK00573AON6M3W5TYWWE6Q: 17
AGFQKJKXKIMUYSK462UTSDU2Q7QQ: 17
AHQIY3JCKATNBTZVZAQLHPDE5KTA: 17

AH3XWQAKGWZVH2IZ2TZYUZLGGWZQ: 17
AGTVZ7ZSDMTEDLMJGZRC7RFJNDWQ: 18
AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: 18
AHX2B4DEER2QR3IU3CCNB3CWC6TA: 18
AHVTPXABLBUBOHIQYQ3QQG6WR6LA: 18
AFKBZVPC42DGI ECDEAOCL57MRAQA: 18
AE23ZFVUOMPKR57BVSWXV34QLMVA: 18
AGAAUEJ3GP5HNRCP54UMGKPNWVUA: 18
AEHPI7DGK45GM4J04PP25BE3RZJQ: 18
AF5MQN46S6QJ3R7F5TEXLMDML66Q: 18
AGK24I67HJ0562FE5RKCGI24T3HA: 18
AE6GRTPFO4YSJUB42J5ID6NUQ26A: 18
AHGYSJ5C6F7YUTGNX6EMVI5QLYUA: 18
AGUG7ONCKL4JOYS6AILUT3AGOCWQ: 18
AEY67UM6R2R2XFEAXAXMWTZ4Z5UA: 18
AGG2QQ2TPFEBHWNTVY05ICVDGHCQ: 18
AE3MLVJ200Z3WLVKQG42G3AQ2KZQ: 19
AHOBK BVHEAT4KPRVI7TYTF6MEJXA: 19
AETEDS6HQJWX2EQYEQFUIAHTTJXQ: 19
AEQWIQ5K6LLNVFPBJTPKK4J76SWA: 19
AETAVHERYX703ND5SFSQ3UZNQ6LQ: 19
AGJ4GTDNTPHMRM26V46EYAF7IGCA: 19
AEPGWR4J3CQI76JJVCCOQQJEH2CA: 19
AGIOGJDBPAJSQTMQRYQMNUVRZ60A: 19
AHIPKIT25JKKXMUL332SVAWYAKBA: 19
AERE4WE4ZXZUK2TWOR33P6XWHPHQ: 19
AGXCV25HDW4JXI5QQSJTIZUHPRHQ: 20
AHSLFXG4FNPYB567N4G2TZH6SK5A: 20
AH26RPVEVUPMVDCL00FLVAIOYHFQ: 20
AGXH2GDBRITH4C3BCMUJY2H4LCIA: 20
AGUQUN4TIWP46MPNHP5QOPOP RPFA: 20
AFFMTXZ6ZKG7PV3B06UKZBSLXYP A: 20
AFEG7SPWPJAFUDLSOGA3QNU6XELA: 20
AEAL4MJGXLIH3VY2GEZJUT54KILA: 20
AGWYFSFSBGELWEVIJSTNQFP5QOZQ: 20
AH2KNTQFU4DWMQIMTDLFSMRQHRYA: 20
AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: 21
AGX2NNH2M3RNS5SODCZMONPR2PAA: 21
AG3S4FROO422V5KP7DJCBXVUQLJQ: 21
AGUEUIXS3ETYDF3UKLAP42NXEYWQ: 21
AHGDGGMCSMMALHTX6WGLJFFUVJXA: 21
AE7GVDNRY5NABEP253Y3GMDUZ0ZA: 21
AG6EDKW4LTUP3EKVJUFQWJTF3AYA: 21
AF2TXYUSL64QAUAQG5H54JHCGSKA: 21
AF7DTXFZKRXELYXQKP54UAHBPDDA: 21
AHLFYDRLX6L6LLJ4L25C5XFNEWKA: 21
AFQGYW6TLQ7IOG475GG6WDXHPY5Q: 21
AFPDU4XGTLCRX5E6OUTZQFDALVTQ: 21

AEB5MUKIPEWYQ6TIWV2WQJK3XVYA: 21
 AHQKCRAIW6KOCBXNQQ2HCMDHPPCQ: 21
 AGKIBYKFLBFAZ2LELCHNWU3JKPCA: 22
 AHCIO4Y7MASS5GHGAZNOPPTXFTVA: 22
 AFMOXNCYTYVRTZKJGVX4X7NHOU7Q: 22
 AG5BM4MB6I32TDPE2SCSQXSCJCHQ: 22
 AGBDLSGHRFYWVTPVFFMGGT3W4NNA: 22
 AGH7F3VLUYMLE5207FHD7X2B5RAA: 22
 AFBFEQ63CH3PADWCT52FIYQLG7XA: 23
 AETDJT3WINHMRGVYOPS7SF7KLHVQ: 23
 AG26EFNGF4Y2N3LC7XZRNLTG4P4A: 23
 AEWFRZ4QAEFDQEBPTVMJ600SIH2Q: 23
 AFFELFYTX2FJCTZLASZLK3K7FQNA: 23
 AEVIDOETQTYGFUVJGTIDMWH4YFQ: 24
 AERJUD4NREPMW7VM26CSV32KJZJQ: 24
 AEJSQT3KBWNDGDU2NJI EPKNTGLMA: 24
 AHMY4433RXWWIPL52FFK3KA2BA: 24
 AG42JLA6HBMONDJECN3JLAUY3JXA: 25
 AECRFIY3HZZG5KMU424HTLAKDGFA: 25
 AF4H7MSDHSUVEMA2BPLP6MWGITGQ: 25
 AG44UYFDZHI6FRBQSLDWOGIEOY4A: 26
 AHDJSYNKXNWW2MQQPABYYMH7RTRQ: 26
 AEKHOFU35JS3BOIWZSEM7DKSTEVA: 27
 AHMQXXMPUH52LC6PKXH4YAJT2FDQ: 27
 AH05IPKZNXFFG2LYSCTINORZG7EQ: 27
 AGBWDIAHWZGV2SIYX2KDCBB6746A: 27
 AEF57CDFWDBRC4QJ3RPLBAHB2JGA: 27
 AG3PQBC2VIMREI2WXU7IFYRIGTPQ: 28
 AFYQUX7UGA5EIGT6SIU6GT5Q67MQ: 29
 AF3NTN5FD5C3WWBBL4EYNZPA7GVA: 29
 AEHWHU26Z2VEZQY3IEEW5HBIGX5Q: 30
 AHLW46IEEWKFIKZQIWJME267TRZQ: 30
 AEKUZFUA5Y7YXZTA3W3L3SHLQCQ: 32
 AH4GZFH3BWFUJTLJ7P4QEFSWGZYA: 32
 AHEMAURZYRCARVTVF6CQYTYE4I4Q: 32
 AFWEOWK32WTIBX4MC7D3WFDLCKJQ: 33
 AFDAU5M5NRUN4LYLI7N4YROKYLWA: 34
 AHNG3NRGEDYO7E44QMYDHMBADPBA: 35
 AHJQIRE3KBRMPLVEGQNUOTYILZDA: 36
 AG6ATLCKETNYRKL6GCQV35CSSGPA: 36
 AHR6CSQFJVG5AHI25CP4NTJDEZfq: 38
 AG4ZLTRHVAZRU33BPY5Y643IZXPQ: 40
 AGVT5E2C6WNOCFYSLOCPAPBJOIKQ: 41
 AG5ZVXXHEXDYUUDSEQC4XXV7DPA: 43
 AFJQQW06VAZCTTY3GXNB3SEUU34Q: 43
 AF7CC34DK36SQJS7WXI44DREGWJA: 48

Compute Hit rate on the test set for the 2 groups of users and discuss the difference (or similarity)

between those scores.

```
[49]: # Restrict ground_truth to the two user groups (from cell 20)
gt_top = {user: items for user, items in ground_truth.items() if user in
    ↳top_users}
gt_bottom = {user: items for user, items in ground_truth.items() if user in
    ↳bottom_users}

# Compute the metrics for each group using the existing evaluate function.
# Here, we extract the hit rate (index 0 of the returned tuple).

svd_top_metrics = evaluate(svd_recs, gt_top, k=10, total_items=total_items)
svd_bottom_metrics = evaluate(svd_recs, gt_bottom, k=10,
    ↳total_items=total_items)
print("SVD Hit Rate for Top 20% Users:", svd_top_metrics[0])
print("SVD Hit Rate for Bottom 20% Users:", svd_bottom_metrics[0])

# Compute metrics for KNNWithMeans on top and bottom users.
knn_top_metrics = evaluate(knn_recs, gt_top, k=10, total_items=total_items)
knn_bottom_metrics = evaluate(knn_recs, gt_bottom, k=10,
    ↳total_items=total_items)
print("KNN Hit Rate for Top 20% Users:", knn_top_metrics[0])
print("KNN Hit Rate for Bottom 20% Users:", knn_bottom_metrics[0])

# Compute metrics for TopPop on top and bottom users.
top_pop_top_metrics = evaluate(top_pop_recs, gt_top, k=10,
    ↳total_items=total_items)
top_pop_bottom_metrics = evaluate(top_pop_recs, gt_bottom, k=10,
    ↳total_items=total_items)
print("TopPop Hit Rate for Top 20% Users:", top_pop_top_metrics[0])
print("TopPop Hit Rate for Bottom 20% Users:", top_pop_bottom_metrics[0])
```

```
SVD Hit Rate for Top 20% Users: 0.09836065573770492
SVD Hit Rate for Bottom 20% Users: 0.11949685534591195
KNN Hit Rate for Top 20% Users: 0.18032786885245902
KNN Hit Rate for Bottom 20% Users: 0.14465408805031446
TopPop Hit Rate for Top 20% Users: 0.16393442622950818
TopPop Hit Rate for Bottom 20% Users: 0.3710691823899371
```

Sort the items by number of ratings in the train set and select the top 20% and last 20% of items.

```
[50]: # Calculate the number of ratings for each item in the trainset
item_rating_counts = {}
for inner_iid, user_ratings in trainset.ir.items():
    raw_iid = trainset.to_raw_iid(inner_iid)
    item_rating_counts[raw_iid] = len(user_ratings)

# Sort items by the count of ratings (ascending order)
```



```

sorted_items = sorted(item_rating_counts.items(), key=lambda x: x[1])
num_items = len(sorted_items)
num_20_percent = int(num_items * 0.2)

# Select bottom 20% (items with fewest ratings)
bottom_20_items = [item for item, count in sorted_items[:num_20_percent]]

# Select top 20% (items with most ratings)
top_20_items = [item for item, count in sorted_items[-num_20_percent:]]

print("Total number of items:", num_items)
print("Bottom 20% items:", bottom_20_items)
print("Top 20% items:", top_20_items)

```

Total number of items: 509

Bottom 20% items: ['B07KRZ1HH8', 'B0BNCDVD4M', 'B0B2LMZ9RT', 'B0C5CTX5LZ', 'B07DVX841V', 'B075MCRFYZ', 'B08Y6PVK2L', 'B073RTT48C', 'B08PBNXZYP', 'B0B63DPVDK', 'B00GGMWA50', 'B07YMGVPV7P', 'B0BGQZK53', 'B09JYJYK1N', 'B00ABLI0IC', 'B01LEZVYZ0', 'B09Y5NZ8PN', 'B07BFZ2KSJ', 'B0C6HXY3R', 'B07M6B3RDW', 'B07MDT5WJX', 'B01FM8IN3U', 'B01HUCYOXE', 'B078JQV1DN', 'B07HGRFG5J', 'B07BRK2M2Q', 'B09Y1X5BSC', 'B0BFR5LN7M', 'B0BT2Q355P', 'B086PLHS9L', 'B00X7DDGBW', 'B09B45DLQG', 'B0002MSQVQ', 'B09NLV5LBK', 'B0BJF97MDV', 'B0776RBZF1', 'B00CIHB8FY', 'B07RLVH6H4', 'B08T9CCXZB', 'B07ND5PQTP', 'B00HU25WSQ', 'B0192JRA2A', 'B07VPLFNYM', 'B01M5CXH65', 'B01LYJMH9P', 'B0BJ8WDC13', 'B09QZJ57YW', 'B01LKYTQOK', 'B0777KT21W', 'B0C1Y5KJPX', 'B0035LCFRW', 'B000EEL50Q', 'B09G5KDVWW', 'B07Y94MSG9', 'B000PR3JEM', 'B0002D0MI0', 'B0158GWWJ2', 'B0085545Y0', 'B0BJ3BBTXH', 'B09JMYM88H', 'B00724YM7E', 'B092LF6RLS', 'B00LYFFW90', 'B01LL8PRIY', 'B01LZBHHC2', 'B076ZSHQ47', 'B0B81FBRNV', 'B0756P1DV8', 'B079NS31NK', 'B09PVPJVN8', 'B06XKPHXS', 'B00BESRB8Q', 'B0B52FBC66', 'B0BQ4HSC9', 'B00YJ5C46K', 'B00W7ZL4TM', 'B00GK75WXC', 'B07B16JL73', 'B006MVECVE', 'B00IPHISW2', 'B076VWK8JV', 'B07RYQ1X42', 'B0B8P1YW7L', 'B0B8Z2T4PS', 'B01JIQ36DE', 'B0C3QWRH66', 'B0BXT6TM6N', 'B0BT3B7GTY', 'B07KDYH2DF', 'B074QKLW4N', 'B09PJ3N73Y', 'B000TGSM6E', 'B0002D0Q2W', 'B0009DXEEM', 'B09M7DPHCS', 'B0BLLKKFR8', 'B07B4S63NN', 'B0002KZEAE', 'B0B8Z6YPWN', 'B08VJC3GCM', 'B079Y9L1G1']

Top 20% items: ['B078KTV8L4', 'B0BR2G8WYG', 'B015WUVA5G', 'B07SZLTL1W', 'B0BT2ZRCY2', 'B075MPFFRV', 'B09PBL98Q4', 'B09YDBKT7M', 'B0BPKTYTPQ', 'B08DM9NFLH', 'B096K2LZKH', 'B09KWS7DH', 'B015QK3GU0', 'B01DXVB4RU', 'B06X9KG3HW', 'B0BXT384GR', 'B00JL5I61A', 'B001L8NGJ2', 'B07V46KRD8', 'B00HZH4D04', 'B0C61YVW2S', 'B07N2HQ1T7', 'B07226MCY6', 'B07JGCQYN8', 'B01ANNKMI', 'B01MRJUB7Z', 'B09396NY1C', 'B0BH599FJY', 'B0BPKQ9QXD', 'B000RNB720', 'B0010SHU18', 'B06W2NS7RW', 'B07D5W5X3Z', 'B07MHL48V6', 'B0BPKH4HB2', 'B07N5J3GTB', 'B00CPL0DUU', 'B015X3CXXA', 'B07TDNWYSB', 'B0B8F6LD9F', 'B06XB3FQKB', 'B07GFH8NTB', 'B08DBDD6CM', 'B09KR857F4', 'B0BPKKVQZT', 'B0B8Z76J32', 'B0B8M5FJ9W', 'B00NARHNCS', 'B091MZPYBQ', 'B005MOCK9M', 'B008L3SXE8', 'B09S2VB3KW', 'B00H4PEMM6', 'B097FMTZCP', 'B0BK5CXZBT', 'B01DK40R3M', 'B0B92LPGD2', 'B098KXQJYV', 'B095XZJ99J',

```
'B00CGFRJ2Y', 'B0BG95DG2H', 'B097Q9W8MW', 'B015IJI05U', 'B07CBT4SYD',
'B08BW2TBD7', 'B09PWC6T9', 'B09M7P8SBH', 'B008BPI20W', 'B012VQ5A7S',
'B09BF8XDF4', 'B01DE4DTAG', 'B079TLFL33', 'B0C5PRQ9K6', 'B01DE4DPSC',
'B00RVE9X06', 'B074WXM13T', 'B00WHLLDWO', 'B0BKZ3GCGR', 'B00H35YIJE',
'B09RJZSXML', 'B00646MZHK', 'B0928HW2P4', 'B01DPCONFM', 'B00IFOTSJW',
'B0B8M2Q1DS', 'B0BHS5C3G', 'B0BKR2ZM9X', 'B0BPLFP5P6', 'B0179A4ILK',
'B01K1EJQ9U', 'B08JMR2JK', 'B09V91H5XM', 'B004XNK7AI', 'B0B95V41NR',
'B08SJY4T7K', 'B0BTC9YJ2W', 'B08R5GM6YB', 'B0BCK6L7S5', 'B09857JRP2',
'B0BPJ4Q6FJ', 'B0BSGM6CQ9']
```

Compute Coverage on the test set for the 2 groups of items and discuss the difference (or similarity) between those scores.

```
[51]: # Function to compute coverage for a given recommender's recommendations.
def compute_coverage(recs, group_top, group_bottom, model_name):
    rec_top = set()
    rec_bottom = set()
    for user in test_users:
        rec_list = recs.get(user, [])
        rec_top.update(set(rec_list).intersection(group_top))
        rec_bottom.update(set(rec_list).intersection(group_bottom))
    coverage_top = len(rec_top) / len(group_top) if group_top else 0
    coverage_bottom = len(rec_bottom) / len(group_bottom) if group_bottom else 0
    print(f"{model_name} Coverage for Top 20% Items: {coverage_top:.4f}")
    print(f"{model_name} Coverage for Bottom 20% Items: {coverage_bottom:.4f}\n")

# Compute and print coverage for each model.
compute_coverage(svd_rec, top_20_items, bottom_20_items, "SVD")
compute_coverage(knn_rec, top_20_items, bottom_20_items, "KNNWithMeans")
compute_coverage(top_pop_rec, top_20_items, bottom_20_items, "TopPop")
```

```
SVD Coverage for Top 20% Items: 0.1881
SVD Coverage for Bottom 20% Items: 0.0198
```

```
KNNWithMeans Coverage for Top 20% Items: 0.4752
KNNWithMeans Coverage for Bottom 20% Items: 0.6238
```

```
TopPop Coverage for Top 20% Items: 0.0990
TopPop Coverage for Bottom 20% Items: 0.0000
```

Error analysis for the neighborhood-based CF Select two users from the training set, one with high RR and one with low RR. These are your reference users. Retrieve the 10 nearest neighbours of each reference user. Print their rating history and analyse their predictions. How much overlap is there between the rating history of the reference users and their neighbours?

For those users or items that your model performs poorly on (RR = 0.05), discuss the potential reasons behind.

```
[52]: # Compute per-user Reciprocal Rank (RR) for the KNN model using the
      ↪ recommendations in knn_recs
user_rr = {}
for user, true_items in ground_truth.items():
    rec = knn_recs.get(user, [])
    rr = 0
    for i, item in enumerate(rec):
        if item in true_items:
            rr = 1.0 / (i + 1)
            break
    user_rr[user] = rr

# Sort users by RR (highest first)
sorted_rr = sorted(user_rr.items(), key=lambda x: x[1], reverse=True)
print("Top 5 users with highest RR:", sorted_rr[:5])
print("Top 5 users with lowest RR:", sorted_rr[-5:])

# Select one reference user with highest RR and one with lowest RR
high_rr_user = sorted_rr[0][0]
low_rr_user = sorted_rr[-1][0]
print(f"\nSelected high RR user: {high_rr_user} (RR = {user_rr[high_rr_user]})")
print(f"Selected low RR user: {low_rr_user} (RR = {user_rr[low_rr_user]})")

# Function to get the k nearest neighbors (using knn_best of the
      ↪ neighborhood-based model)
def get_neighbors(user_id, k=10):
    inner_uid = trainset.to_inner_uid(user_id)
    neighbor_inner = knn_best.get_neighbors(inner_uid, k=k)
    return [trainset.to_raw_uid(nid) for nid in neighbor_inner]

high_rr_neighbors = get_neighbors(high_rr_user, k=10)
low_rr_neighbors = get_neighbors(low_rr_user, k=10)
print(f"\nHigh RR user neighbors: {high_rr_neighbors}")
print(f"Low RR user neighbors: {low_rr_neighbors}")

# Function to obtain the rating history of a user from the training set
def get_rating_history(user_id):
    inner_uid = trainset.to_inner_uid(user_id)
    # trainset.ur is a dict: key is inner_uid, value is list of (inner_iid,
    ↪ rating)
    history = trainset.ur[inner_uid]
    return [(trainset.to_raw_iid(iid), rating) for iid, rating in history]

# Print rating history for each reference user and their neighbors
print("\nHigh RR user rating history:")
high_rr_history = get_rating_history(high_rr_user)
print(high_rr_history)
```

```

print("\nHigh RR user neighbors' rating histories:")
for neighbor in high_rr_neighbors:
    print(f"Neighbor {neighbor}: {get_rating_history(neighbor)}")

print("\nLow RR user rating history:")
low_rr_history = get_rating_history(low_rr_user)
print(low_rr_history)

print("\nLow RR user neighbors' rating histories:")
for neighbor in low_rr_neighbors:
    print(f"Neighbor {neighbor}: {get_rating_history(neighbor)}")

# Overlap analysis: for each neighbor compare the set of rated items with the
↪reference user.
def compute_overlap(ref_history, neighbor_history):
    ref_items = set(item for item, _ in ref_history)
    neighbor_items = set(item for item, _ in neighbor_history)
    common = ref_items.intersection(neighbor_items)
    return len(common), len(ref_items), len(neighbor_items), common

print("\nOverlap analysis for high RR user:")
high_total_overlap = 0
for neighbor in high_rr_neighbors:
    overlap, ref_count, neigh_count, common_items =
↪compute_overlap(high_rr_history, get_rating_history(neighbor))
    print(f"Neighbor {neighbor}: {overlap} common items (Reference rated
↪{ref_count}, Neighbor rated {neigh_count})")
    high_total_overlap += overlap
high_avg_overlap = high_total_overlap / len(high_rr_neighbors)
print(f"\nAverage overlap for high RR user with neighbors: {high_avg_overlap:.
↪2f}")

print("\nOverlap analysis for low RR user:")
low_total_overlap = 0
for neighbor in low_rr_neighbors:
    overlap, ref_count, neigh_count, common_items =
↪compute_overlap(low_rr_history, get_rating_history(neighbor))
    print(f"Neighbor {neighbor}: {overlap} common items (Reference rated
↪{ref_count}, Neighbor rated {neigh_count})")
    low_total_overlap += overlap
low_avg_overlap = low_total_overlap / len(low_rr_neighbors)
print(f"\nAverage overlap for low RR user with neighbors: {low_avg_overlap:.
↪2f}")

```

Top 5 users with highest RR: [('AHGDGGMCSMMALHTX6WGLJFFUVJXA', 1.0),
 ('AHN535MBJTI4BGLTUORFR3TLABLA', 1.0), ('AFJ52QWR6Y5WPXK3MEZJJJBHMUY7A', 1.0),

('AFMOXNCYTYVRTZKJGVX4X7NHOU7Q', 1.0), ('AH2KNTQFU4DWMQIMTDLFMSRQHRYA', 0.5)]
Top 5 users with lowest RR: [('AG5ZVXXHEXDYUUDSEQC4XXV7DPA', 0),
('AFKZTCN4BNEKHAJQARBTQ4RZVETQ', 0), ('AH5MZVAOBU2PWPJ2G663BY5S2R5A', 0),
('AEIRBLA7WN4R3DFFWOUBDY6UYWEA', 0), ('AENTQR2OEQ07IPIVUMHVMV545FPJA', 0)]

Selected high RR user: AHGDGGMCSMMALHTX6WGLJFFUVJXA (RR = 1.0)

Selected low RR user: AENTQR2OEQ07IPIVUMHVMV545FPJA (RR = 0)

High RR user neighbors: ['AE5M7M2VYMM3WHJIBLCHHEU7UOXQ',
'AESDNHQRREZWTZF7A7TFFFTSNY5Q', 'AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA',
'AG4VWLJ2X7PIBYDXYUNFY5ONDRQ', 'AEBXDQP3DYUQMQUATAUIMYMOMSRZQ',
'AEDKV2KYMJZD43MAADUEKE7Z3PPA', 'AHJPWPHCWZ5MX2UY6CNGJUNBWP3A',
'AE00BBNDQVLYBDVJLE4GJUCWX42A', 'AGXCV25HDW4JXI5QQSJITIZUHPRHQ',
'AENYK6CKXCHJDY2YV3WIQZMJ3HNQ']

Low RR user neighbors: ['AE7P3HIBI3UDLCJLUPUZQWYVPEWA',
'AESDNHQRREZWTZF7A7TFFFTSNY5Q', 'AGX2NNH2M3RNS5SODCZMONPR2PAA',
'AEKUZFUOA5Y7YXZTA3W3L3SHLQCQ', 'AGXCV25HDW4JXI5QQSJITIZUHPRHQ',
'AEYJZUNG4OSYKDPN7WH3LKTBKH3Q', 'AFVLYQWBCIHXVLH6X2FSXRMAHHXA',
'AH5MZVAOBU2PWPJ2G663BY5S2R5A', 'AER6WLSJPBPNOZIQVWHQBVOKTNCA',
'AFBIB5XMB27ZIUM3YDTEOKSLLQA']

High RR user rating history:

[('B00646MZHK', 5.0), ('B0085545Y0', 5.0), ('B08R5GM6YB', 5.0), ('B000EEL50Q', 4.0), ('B01DECWM0G', 1.0), ('B008BPI20W', 5.0), ('B01DE4DEGK', 5.0),
('B0BFKKVQZT', 5.0), ('B0BNVFLP64', 5.0), ('B095XCZXNQ', 3.0), ('B09857JRP2', 5.0), ('B07FZ5XBCJ', 5.0), ('B00GGMWA50', 5.0), ('B00YJ5C46K', 5.0),
('B0027V760M', 5.0), ('B0150YC54E', 4.0), ('B00X7DDGBW', 5.0), ('B01J5A060U', 5.0), ('B09JYHK3KF', 5.0), ('B007MY5BDI', 5.0), ('B07KDYH2DF', 5.0)]

High RR user neighbors' rating histories:

Neighbor AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: [('B000KIPTE4', 5.0), ('B005MOCK9M', 5.0), ('B005MOMUQK', 5.0), ('B007T80GLK', 5.0), ('B097Q9W8MW', 4.0),
('B09JYHK3KF', 5.0)]

Neighbor AESDNHQRREZWTZF7A7TFFFTSNY5Q: [('B0C67HCGBR', 5.0), ('B00HZH4D04', 4.0), ('B0C61YVW2S', 5.0), ('B0BNVFLP64', 4.0), ('B0B95V41NR', 4.0),
('B00HU260SW', 4.0), ('B08SJY4T7K', 4.0), ('B000T9L7W2', 5.0), ('B06W2NS7RW', 4.0), ('B0BPJ4Q6FJ', 5.0), ('B07DK59QNR', 5.0), ('B07N2HQ1T7', 5.0),
('B07D5W5X3Z', 4.0), ('B07NP7HF7X', 4.0), ('B00JL5I61A', 4.0), ('B016QUXTZU', 4.0), ('B0BZ1XQX97', 4.0)]

Neighbor AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: [('1423414357', 5.0), ('B07JCW1KGG', 5.0), ('B000T9L7W2', 5.0), ('B000RNB720', 5.0), ('B0BR2Q367S', 5.0),
('B0BPJ4Q6FJ', 5.0), ('B0BPKTYTPQ', 5.0), ('B005T800V2', 5.0), ('B00646MZHK', 5.0), ('B074KRRZCH', 3.0), ('B01MRJUB7Z', 5.0), ('B0BFKKVQZT', 5.0),
('B09PWC6T9', 5.0), ('B07B16JL73', 5.0), ('B081592T7F', 5.0), ('B00DKAQGTG', 5.0), ('B095QBH5ZB', 5.0), ('B07DWM2XSS', 5.0), ('B07PM289SV', 5.0),
('B00D35CNL8', 5.0), ('B0817CVBLV', 5.0)]

Neighbor AG4VWLJ2X7PIBYDXYUNFY5ONDRQ: [('B0C61YVW2S', 4.0), ('B0BKZ5F8BS', 5.0), ('B09BR11LYJ', 5.0), ('B0BCK6L7S5', 5.0), ('B019NY2PKG', 5.0),

('B07FZ5XBCJ', 5.0), ('B07N5J3GTB', 5.0)]

Neighbor AEBXDQP3DYUQMQUATAUIMYOMSRZQ: [('B000TGSM6E', 2.0), ('B07DWLYGKH', 5.0), ('B003BGXW50', 4.0), ('B01CSAATY6', 4.0), ('B078L11275', 5.0), ('B09857JRP2', 3.0), ('B097FMTZCP', 4.0), ('B0002D0Q2W', 2.0), ('B0B8F6LD9F', 5.0), ('B076ZSHQ47', 4.0), ('B095QBH5ZB', 5.0)]

Neighbor AEDKV2KYMJZD43MAADUEKE7Z3PPA: [('B000T9L7W2', 3.0), ('B0C61YVW2S', 5.0), ('B0B8M2Q1DS', 5.0), ('B0BTC9YJ2W', 5.0), ('B0002E3FBA', 5.0), ('B01DE4DEGK', 5.0), ('B0BSGM6CQ9', 4.0), ('B091MZPYBQ', 5.0), ('B07N5J3GTB', 5.0), ('B08R5GM6YB', 5.0), ('B06XB3FQKB', 5.0), ('B000YUSHJS', 4.0)]

Neighbor AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: [('B0BPJ4Q6FJ', 4.0), ('B004U1QDLO', 4.0), ('B06XB3FQKB', 2.0), ('B00G964BUY', 5.0), ('B0BXT41XC6', 5.0), ('B07M69RBH7', 5.0), ('B0150YC54E', 5.0), ('B004XNK7AI', 5.0), ('B0B8YX1H2H', 5.0), ('B00CI06BEA', 2.0), ('B0BT2W3TTM', 5.0), ('B00WT7KPCK', 5.0), ('B0C5PRQ9K6', 5.0), ('B07NKMZJ5M', 5.0), ('B0BSR996X8', 4.0), ('B005MOKLGQ', 3.0), ('B09PBL98Q4', 3.0), ('B0BCK6L7S5', 5.0)]

Neighbor AEO0BBNDQVLYBDVJLE4GJUCWX42A: [('B0BFKKVQZT', 4.0), ('B08R5GM6YB', 4.0), ('B08SJY4T7K', 4.0), ('B07XP6MHQT', 5.0), ('B0010SHU18', 3.0), ('B07V46KRD8', 5.0), ('B01CS87RGQ', 5.0), ('B0BXT384GR', 2.0), ('B00IPH8MD2', 5.0), ('B0B95V41NR', 3.0), ('B01NAZMWZK', 1.0), ('B07N5J3GTB', 5.0), ('B00IZA1GI2', 5.0)]

Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: [('B0002D0Q2W', 5.0), ('B079P9LDHN', 5.0), ('B00IFOTSJW', 5.0), ('B00JL5I61A', 3.0), ('B01DZ6FBB1', 4.0), ('B09RJZSQML', 5.0), ('B00P6ZOPP0', 3.0), ('B0BPLFP5P6', 4.0), ('B0BCK6L7S5', 4.0), ('B007T8CUNG', 4.0), ('B012VQ5A7S', 5.0), ('B01DK40R3M', 4.0), ('B004U1QDLO', 4.0), ('B0040K1G64', 5.0), ('B01MRJUB7Z', 4.0), ('B09TRX519R', 3.0), ('B081592T7F', 5.0), ('B01DECWM0G', 4.0), ('B0BSGM6CQ9', 5.0), ('B073T2YK2J', 5.0)]

Neighbor AENYK6CKXCHJDY2YV3W1QZMJ3HNQ: [('B00IFOTSJW', 5.0), ('B00646MZHk', 5.0), ('B004XNK7AI', 5.0), ('B07CBT4SYD', 5.0), ('B0BYXVCSGG', 5.0), ('B06Y6BCX7B', 5.0), ('B0B3VSZQHL', 5.0), ('B079P9LDHN', 5.0), ('B000CZORKG', 5.0), ('B00UTD0YG8', 5.0)]

Low RR user rating history:

(('B09TRX519R', 5.0), ('B00BESRB8Q', 4.0), ('B0BHSG5C3G', 4.0), ('B00UTELPGK', 4.0), ('B06W2NS7RW', 4.0), ('B0BT2Z1DF1', 5.0), ('B00HY657E2', 4.0), ('B0B29LK8QD', 5.0), ('B0BLZD2X18', 5.0), ('B014ZYPT34', 5.0), ('B00H35YIJE', 5.0), ('B07KYX3S8X', 5.0)]

Low RR user neighbors' rating histories:

Neighbor AE7P3HIBI3UDLCJLUPUZQWYVPEWA: [('B09BF8XDF4', 4.0), ('B074WXM13T', 5.0), ('B07GFH8NTB', 5.0), ('B00H35YIJE', 4.0), ('B07B4S63NN', 4.0), ('B003AYNHGW', 5.0), ('B00ONGVQK0', 4.0), ('B095QBH5ZB', 4.0), ('B07DWM2XSS', 5.0)]

Neighbor AESDNHQRREZWTZF7A7TFFFTSNY5Q: [('B0C67HCGBR', 5.0), ('B00HZH4D04', 4.0), ('B0C61YVW2S', 5.0), ('B0BNVFLP64', 4.0), ('B0B95V41NR', 4.0), ('B00HU260SW', 4.0), ('B08SJY4T7K', 4.0), ('B000T9L7W2', 5.0), ('B06W2NS7RW', 4.0), ('B0BPJ4Q6FJ', 5.0), ('B07DK59QNR', 5.0), ('B07N2HQ1T7', 5.0), ('B07D5W5X3Z', 4.0), ('B07NP7HF7X', 4.0), ('B00JL5I61A', 4.0), ('B016QUXTZU',

4.0), ('B0BZ1XQX97', 4.0)]

Neighbor AGX2NNH2M3RNS5S0DCZMONPR2PAA: [('B079TLFL33', 5.0), ('B00H35YIJE', 4.0), ('B0B95V41NR', 5.0), ('B000RNB720', 3.0), ('B08R5GM6YB', 4.0), ('B07CBT4SYD', 3.0), ('B0B2LSX437', 5.0), ('B0BXT384GR', 3.0), ('B07WW5PKCP', 5.0), ('B0B8Z6GZZ5', 5.0), ('B01DE4DEGK', 5.0), ('B06XB3FQKB', 5.0), ('B005T800V2', 5.0), ('B0BG95DG2H', 5.0), ('B0BKZ3GCGR', 5.0), ('B01DE4DPSC', 5.0), ('B0064RTS0G', 5.0), ('B0C5PRQ9K6', 5.0), ('B00CI06BEA', 5.0), ('B09V91H5XM', 4.0), ('B00646MZHk', 3.0)]

Neighbor AEKUZFUA5Y7YXZTA3W3L3SHLQCQ: [('B00H4PEMM6', 5.0), ('B0BR2Q367S', 4.0), ('B0BXT384GR', 4.0), ('B091MZPYBQ', 5.0), ('B001W99HE8', 5.0), ('B097FMTZCP', 5.0), ('B077GH8LF6', 5.0), ('B015X3CXXA', 5.0), ('B00H35YIJE', 5.0), ('B0B95V41NR', 5.0), ('B0BRSM5XBB', 3.0), ('B0756P1DV8', 5.0), ('B09SNBQCM8', 5.0), ('B01DZ6FBBi', 5.0), ('B07GFH8NTB', 3.0), ('B007MY5BDI', 5.0), ('B08R5GM6YB', 4.0), ('B07CBT4SYD', 4.0), ('B0010SHU18', 5.0), ('B01DK43HMK', 5.0), ('B0002DOCE0', 5.0), ('B09S2VB3KW', 5.0), ('B01DE4DTAG', 5.0), ('B0B81FBRNV', 5.0), ('B0BQ4HSCK9', 5.0), ('B00HU260SW', 5.0), ('B09198262S', 5.0), ('B0002DOCNA', 5.0), ('B0BKR2ZM9X', 5.0), ('B09NWCDC36N', 5.0), ('B00IPH8MD2', 5.0), ('B08BJGBHN7', 5.0)]

Neighbor AGXCV25HDW4JXI5QQSJTIZUHPRHQ: [('B0002DOQ2W', 5.0), ('B079P9LDHN', 5.0), ('B00IFOTSJW', 5.0), ('B00JL5I61A', 3.0), ('B01DZ6FBBi', 4.0), ('B09RJZSQML', 5.0), ('B00P6ZOPP0', 3.0), ('B0BPLFP5P6', 4.0), ('B0BCK6L7S5', 4.0), ('B007T8CUNG', 4.0), ('B012VQ5A7S', 5.0), ('B01DK40R3M', 4.0), ('B004U1QDLO', 4.0), ('B0040K1G64', 5.0), ('B01MRJUB7Z', 4.0), ('B09TRX519R', 3.0), ('B081592T7F', 5.0), ('B01DECWM0G', 4.0), ('B0BSGM6CQ9', 5.0), ('B073T2YK2J', 5.0)]

Neighbor AEYJZUNG40SYKDPN7WH3LKTbKH3Q: [('B0B8M2Q1DS', 5.0), ('B0BPJ4Q6FJ', 5.0), ('B07CRK35NG', 3.0), ('B015HG7F3Q', 1.0), ('B06W2NS7RW', 4.0), ('B09V91H5XM', 4.0), ('B06XB3FQKB', 4.0), ('B08R5GM6YB', 5.0), ('B08JMQR2JK', 4.0), ('B07D5W5X3Z', 4.0), ('B0BSGM6CQ9', 4.0), ('B07BHJH2Y8', 5.0), ('B095XCZXNQ', 5.0)]

Neighbor AFVLYQWBCIHXLH6X2FSXRMAHHA: [('B07ND5PQTP', 5.0), ('B015XE0B70', 5.0), ('B00IFOTSJW', 5.0), ('B07CBT4SYD', 2.0), ('B09BF8XDF4', 5.0), ('B07G37S6S9', 5.0), ('B0BSCFTV2G', 5.0), ('B0BK5CXZBT', 4.0), ('B09TRX519R', 5.0)]

Neighbor AH5MZVA0BU2PWPJ2G663BY5S2R5A: [('B00H35YIJE', 2.0), ('B09M7DPHCS', 5.0), ('B0C5PRQ9K6', 5.0), ('B0B81FBRNV', 4.0), ('B0B8Z6192W', 3.0), ('B0002KZEAE', 5.0), ('B07226MCY6', 5.0), ('B08NVVLB16', 5.0), ('B07DWM2XSS', 1.0), ('B09G5KLKX2', 5.0)]

Neighbor AER6WLSJPBPNOZIQVWHQBVOKTNCA: [('B0BK5CXZBT', 5.0), ('B0BPJ4Q6FJ', 5.0), ('B0BKZ3GCGR', 4.0), ('B08R5GM6YB', 5.0), ('B00IFOTSJW', 5.0), ('B0BCXRXTNY', 5.0), ('B0BSGM6CQ9', 5.0), ('B06XB3FQKB', 5.0), ('B07L6RCDP7', 5.0), ('B06ZZCMP8L', 5.0), ('B0BHSG5C3G', 2.0), ('B09PWC6T9', 5.0)]

Neighbor AFBIB5XMB27ZIUM3YDTESOKSLLQA: [('B00H35YIJE', 2.0), ('B07CBT4SYD', 5.0), ('B07CRK35NG', 5.0), ('B07NKMZJ5M', 5.0), ('B096K2LZKH', 5.0), ('1423414357', 4.0), ('B00646MZHk', 5.0), ('B0002DOCE0', 5.0), ('B093YKLWVN', 5.0), ('B07V46KRD8', 5.0), ('B0BKR2ZM9X', 5.0), ('B09R6KV6QX', 5.0), ('B00HU260SW', 5.0), ('B0C6J1X7TD', 5.0)]

Overlap analysis for high RR user:

Neighbor AE5M7M2VYMM3WHJIBLCHHEU7UOXQ: 1 common items (Reference rated 21,
Neighbor rated 6)
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 1 common items (Reference rated 21,
Neighbor rated 17)
Neighbor AHEJYPLN7DMCYRQ2KCWQ5YIY6JDA: 2 common items (Reference rated 21,
Neighbor rated 21)
Neighbor AG4VWLJ2X7PIBYDXYUNFY5ONDRQ: 1 common items (Reference rated 21,
Neighbor rated 7)
Neighbor AEBXDQP3DYUQMQUATAUIMYMOMSRZQ: 1 common items (Reference rated 21,
Neighbor rated 11)
Neighbor AEDKV2KYMJZD43MAADUEKE7Z3PPA: 2 common items (Reference rated 21,
Neighbor rated 12)
Neighbor AHJPWPHCWZ5MX2UY6CNGJUNBWP3A: 1 common items (Reference rated 21,
Neighbor rated 18)
Neighbor AEOOBBNDQVLYBDVJLE4GJUCWX42A: 2 common items (Reference rated 21,
Neighbor rated 13)
Neighbor AGXCV25HDW4JXI5QQSJTIUHPRHQ: 1 common items (Reference rated 21,
Neighbor rated 20)
Neighbor AENYK6CKXCHJDY2YV3WIIQZMJ3HNQ: 1 common items (Reference rated 21,
Neighbor rated 10)

Average overlap for high RR user with neighbors: 1.30

Overlap analysis for low RR user:

Neighbor AE7P3HIBI3UDLCJLUPUZQWYVPEWA: 1 common items (Reference rated 12,
Neighbor rated 9)
Neighbor AESDNHQRRZEWTZF7A7TFFFTSNY5Q: 1 common items (Reference rated 12,
Neighbor rated 17)
Neighbor AGX2NNH2M3RNS5SODCZMONPR2PAA: 1 common items (Reference rated 12,
Neighbor rated 21)
Neighbor AEKUZFUA5Y7YXZTA3W3L3SHLQCQ: 1 common items (Reference rated 12,
Neighbor rated 32)
Neighbor AGXCV25HDW4JXI5QQSJTIUHPRHQ: 1 common items (Reference rated 12,
Neighbor rated 20)
Neighbor AEYJZUNG40SYKDPN7WH3LKTBK3Q: 1 common items (Reference rated 12,
Neighbor rated 13)
Neighbor AFVLYQWBCIHXVLH6X2FSXRMAHHXA: 1 common items (Reference rated 12,
Neighbor rated 9)
Neighbor AH5MZVA0BU2PWPJ2G663BY5S2R5A: 1 common items (Reference rated 12,
Neighbor rated 10)
Neighbor AER6WLSJPBPNOZIQWVHQBVOKTNCA: 1 common items (Reference rated 12,
Neighbor rated 12)
Neighbor AFBIB5XMB27ZIUM3YDTESOKSLLQA: 1 common items (Reference rated 12,
Neighbor rated 14)

Average overlap for low RR user with neighbors: 1.00

0.0.3 Week 9 [Feb 24 - Mar 2]: Text Representation

In this part, we will work with the text content from the dataset and will apply NLP techniques to represent items and users in a vector space.

1. Note: for this part, you are free to use all or a subset of the metadata file, e.g., only using items that are in the train/test splits or using the first n characters of the text content. Please justify your choice.

```
[53]: import matplotlib.pyplot as plt

# Load the metadata containing item descriptions (if not already loaded)
df_meta = pd.read_csv('Data/metadata.tsv', sep='\t')

# Compute the length of each description (convert to string to avoid NaN issues)
df_meta['desc_len'] = df_meta['description'].astype(str).apply(len)

# Plot a histogram of the description lengths
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Description Lengths in the Training Set')
plt.show()

# Crop descriptions longer than 10000 characters
df_meta['description'] = df_meta['description'].astype(str).apply(lambda x: x[:
↪10000] if len(x) > 10000 else x)
# Recompute description lengths after cropping
df_meta['desc_len'] = df_meta['description'].apply(len)

# Plot histogram of the new description lengths
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Cropped Description Lengths')
plt.show()

total_reviews = len(df_meta)
reviews_lt_2000 = (df_meta['desc_len'] < 2000).sum()
reviews_lt_1000 = (df_meta['desc_len'] < 1000).sum()
reviews_lt_500 = (df_meta['desc_len'] < 500).sum()

perc_lt_2000 = reviews_lt_2000 / total_reviews * 100
perc_lt_1000 = reviews_lt_1000 / total_reviews * 100
perc_lt_500 = reviews_lt_500 / total_reviews * 100
```

```

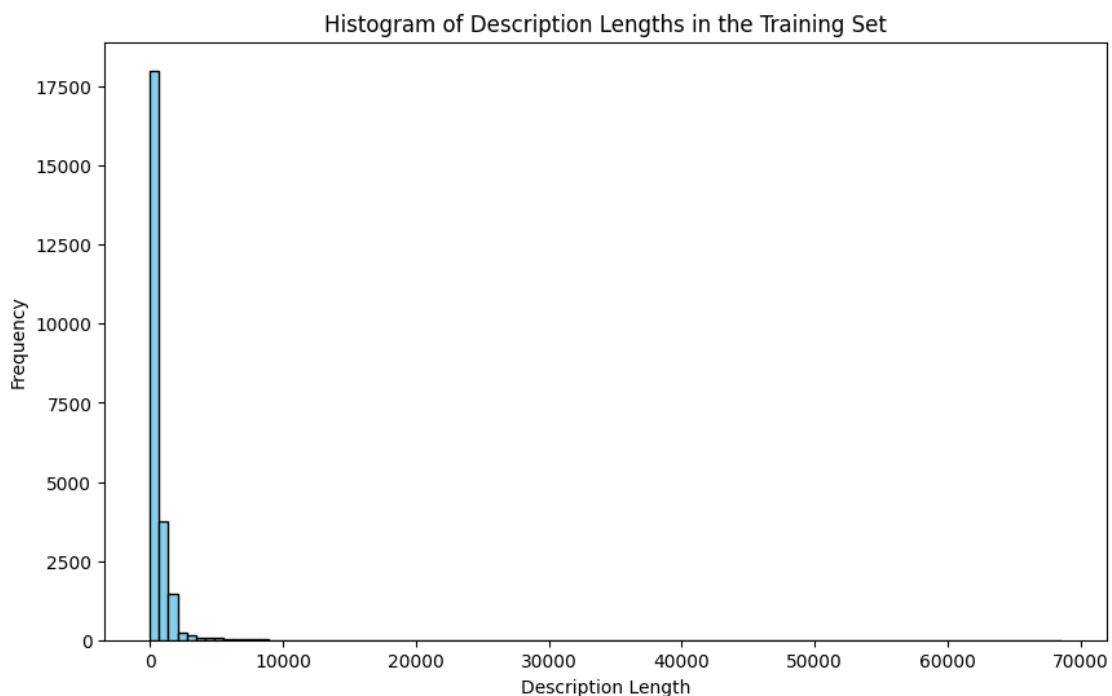
print(f"Percentage of reviews shorter than 2000 chars: {perc_lt_2000:.2f}%")
print(f"Percentage of reviews shorter than 1000 chars: {perc_lt_1000:.2f}%")
print(f"Percentage of reviews shorter than 500 chars: {perc_lt_500:.2f}%")

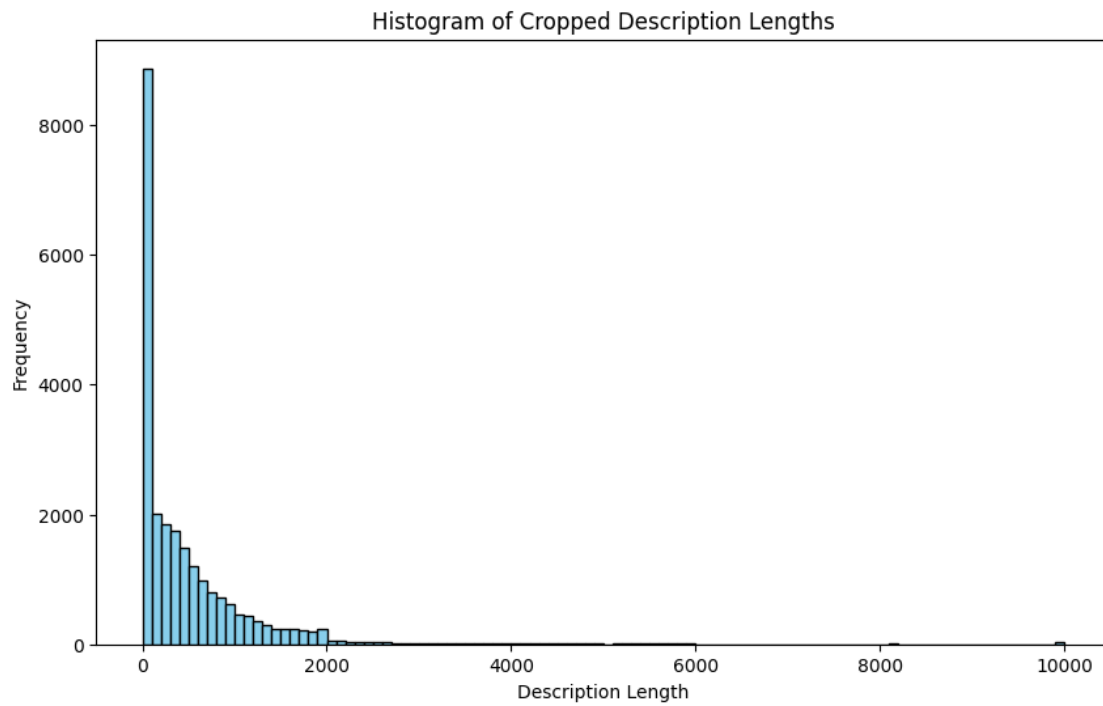
# Crop descriptions to keep only the first 1000 characters
def crop_text(text):
    # If the string is less than or equal to 1000 characters, return it as is.
    if len(text) <= 1000:
        return text
    # Find the last space before the 1000th character.
    cut_index = text.rfind(' ', 0, 1000)
    # If no space is found, break at 1000 characters.
    if cut_index == -1:
        cut_index = 1000
    return text[:cut_index]

df_meta['description'] = df_meta['description'].astype(str).apply(crop_text)
# Recompute description lengths after cropping
df_meta['desc_len'] = df_meta['description'].apply(len)

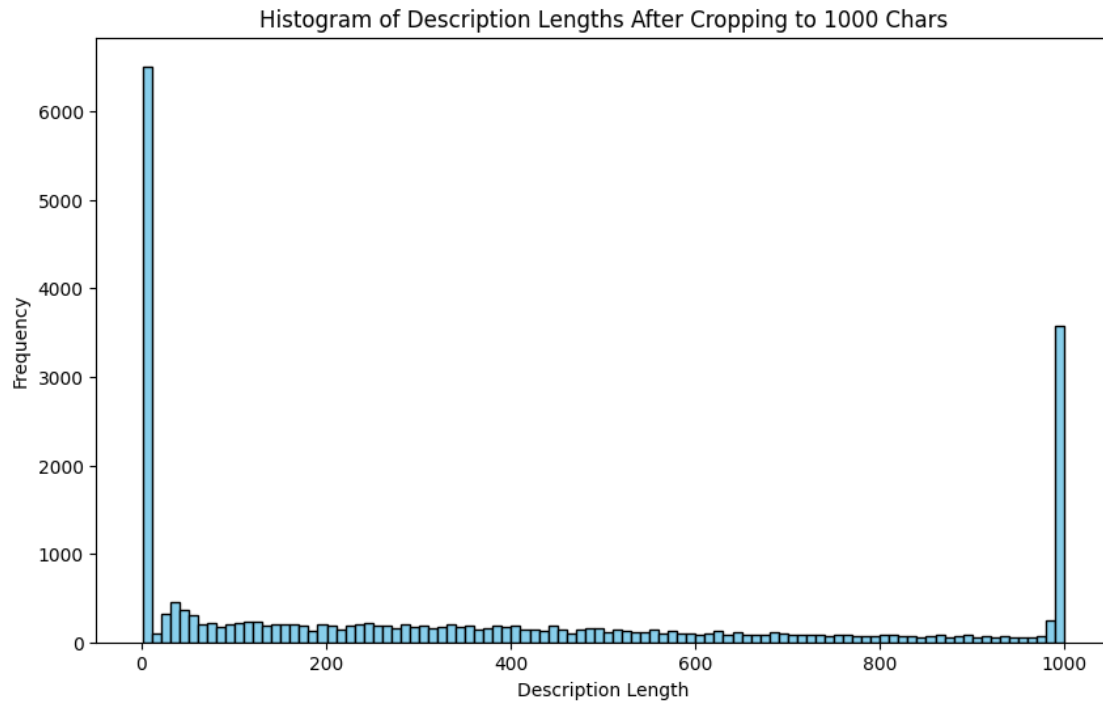
# Plot histogram of the new description lengths after cropping
plt.figure(figsize=(10, 6))
plt.hist(df_meta['desc_len'], bins=100, color='skyblue', edgecolor='black')
plt.xlabel('Description Length')
plt.ylabel('Frequency')
plt.title('Histogram of Description Lengths After Cropping to 1000 Chars')
plt.show()

```





Percentage of reviews shorter than 2000 chars: 96.50%
Percentage of reviews shorter than 1000 chars: 84.48%
Percentage of reviews shorter than 500 chars: 66.51%



2. Select the column description from the metadata file and apply appropriate preprocessing to clean up the data, for example: tokenization, transformation to lowercase, stopword removal, or stemming. Motivate your preprocessing choices and report the vocabulary size before and after preprocessing. There are many libraries you can use, including but not limited to, NLTK, spaCy or CoreNLP (requires Java).

```
[54]: import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import SnowballStemmer
import string

nltk.download('punkt')
nltk.download('stopwords')

# Get the raw descriptions from the metadata DataFrame
descriptions = df_meta['description'].dropna().astype(str).tolist()

# Define stopwords and stemmer
stop_words = set(stopwords.words('english'))
stemmer = SnowballStemmer(language='english')

# Preprocess function: tokenize, lower, remove punctuation, numbers and
# stopwords, and stem
```

```

def preprocess_text(text):
    # Tokenize and lowercase
    tokens = [word.lower() for word in word_tokenize(text)]
    # Remove punctuation tokens, stopwords, and tokens containing any digits
    tokens = [token for token in tokens if token not in string.punctuation and
    ↪ token not in stop_words and not any(ch.isdigit() for ch in token)]
    # Stem tokens
    tokens = [stemmer.stem(token) for token in tokens]
    return tokens

# Build a vocabulary from the raw descriptions (lowercased tokens)
raw_vocab = set()
for text in descriptions:
    tokens = word_tokenize(text.lower())
    raw_tokens = [token for token in tokens if token not in string.punctuation]
    raw_vocab.update(raw_tokens)

# Build a vocabulary from the preprocessed descriptions
processed_vocab = set()
for text in descriptions:
    tokens = preprocess_text(text)
    processed_vocab.update(tokens)

print("Vocabulary size before preprocessing:", len(raw_vocab))
print("Vocabulary size after preprocessing:", len(processed_vocab))

# Create a new column 'preprocessed_description' by applying the preprocessing
↪ function
df_meta['preprocessed_description'] = df_meta['description'].apply(lambda x: ' '
    ↪ '.join(preprocess_text(x)))

# Print a sample original and its preprocessed description
sample_index = 0 # you can change the index to view a different sample
print("Original Description:")
print(df_meta['description'].iloc[sample_index])
print("\nPreprocessed Description:")
print(df_meta['preprocessed_description'].iloc[sample_index])

```

```

[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\david\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\david\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

Vocabulary size before preprocessing: 51296

Vocabulary size after preprocessing: 27243

Original Description:

BEHRINGER EUROPOWER EPQ900. Professional 900-Watt Light Weight Stereo Power Amplifier with ATR (Accelerated Transient Response) Technology. 2 x 390 Watts into 4 Ohms; 2 x 245 Watts into 8 Ohms. 2 x 390 Watts into 4 Ohms; 2 x 245 Watts into 8 Ohms. ATR (Accelerated Transient Response) technology for ultimate punch and clarity. ATR (Accelerated Transient Response) technology for ultimate punch and clarity. Ultra-light, ultra-low noise and ultra-efficient switch-mode power supply for noise-free audio, superior transient response and low power consumption. Ultra-light, ultra-low noise and ultra-efficient switch-mode power supply for noise-free audio, superior transient response and low power consumption. Independent limiters for each channel offer maximum output level with reliable overload protection. Independent limiters for each channel offer maximum output level with reliable overload protection. Detented gain controls for precise setting and matching of sensitivity. Detented gain

Preprocessed Description:

behring europow profession light weight stereo power amplifi atr acceler
transient respons technolog x watt ohm x watt ohm x watt ohm x watt ohm atr
acceler transient respons technolog ultim punch clariti atr acceler transient
respons technolog ultim punch clariti ultra-light ultra-low nois ultra-effici
switch-mod power suppli noise-fre audio superior transient respons low power
consumpt ultra-light ultra-low nois ultra-effici switch-mod power suppli noise-
fre audio superior transient respons low power consumpt independ limit channel
offer maximum output level reliabl overload protect independ limit channel offer
maximum output level reliabl overload protect detent gain control precis set
match sensit detent gain

3. Represent each item in the TF-IDF vector space. You can use your preferred library for text representation, for example, scikit-learn or gensim.

```
[55]: from sklearn.feature_extraction.text import TfidfVectorizer

# Use preprocessed text and split on spaces without additional preprocessing.
tfidf_vectorizer = TfidfVectorizer(tokenizer=lambda x: x.split(),
    ↪lowercase=False)
tfidf_matrix = tfidf_vectorizer.
    ↪fit_transform(df_meta['preprocessed_description'])

print("TF-IDF matrix shape:", tfidf_matrix.shape)
```

```
c:\Users\david\anaconda3\envs\WRS\lib\site-
packages\sklearn\feature_extraction\text.py:517: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
warnings.warn(
```

TF-IDF matrix shape: (23984, 27243)

4. Represent each item using pretrained word embeddings (e.g., GloVe, word2vec).

```
[56]: import os
import gensim
import gensim.downloader as api
import numpy as np

w2v_path = 'Data/GoogleNews-vectors-negative300.bin'

try:
    word2vec = gensim.models.KeyedVectors.load_word2vec_format(w2v_path,
↳binary=True)
except FileNotFoundError:
    print("Downloading word2vec-google-news-300...")
    word2vec = api.load("word2vec-google-news-300")

def get_avg_word2vec(tokens, model):
    vectors = [model[token] for token in tokens if token in model]
    return np.mean(vectors, axis=0) if vectors else np.zeros(model.vector_size)

df_meta['w2v_vector'] = df_meta['preprocessed_description'].apply(lambda txt:
↳get_avg_word2vec(txt.split(), word2vec))

print("Sample word2vec representation (vector shape):", df_meta['w2v_vector'].
↳iloc[0].shape)
```

Downloading word2vec-google-news-300...

Sample word2vec representation (vector shape): (300,)

5. Explore the similarity between items within the vector spaces by computing their cosine similarity. Compare results obtained with TF-IDF and the word embeddings. Discuss what you find. Note that you can select a subset of items to better highlight the differences between the two text representations.

```
[57]: import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Select a subset of items (e.g., the first 10 items with more than 900
↳characters)
subset = df_meta[df_meta['desc_len'] > 900].iloc[:10]

# TF-IDF similarity
subset_tfidf = tfidf_matrix[subset.index, :]
tfidf_sim = cosine_similarity(subset_tfidf)

# Word Embedding similarity: stack the w2v_vector representations into a matrix
subset_w2v = np.stack(subset['w2v_vector'].values)
w2v_sim = cosine_similarity(subset_w2v)
```

```

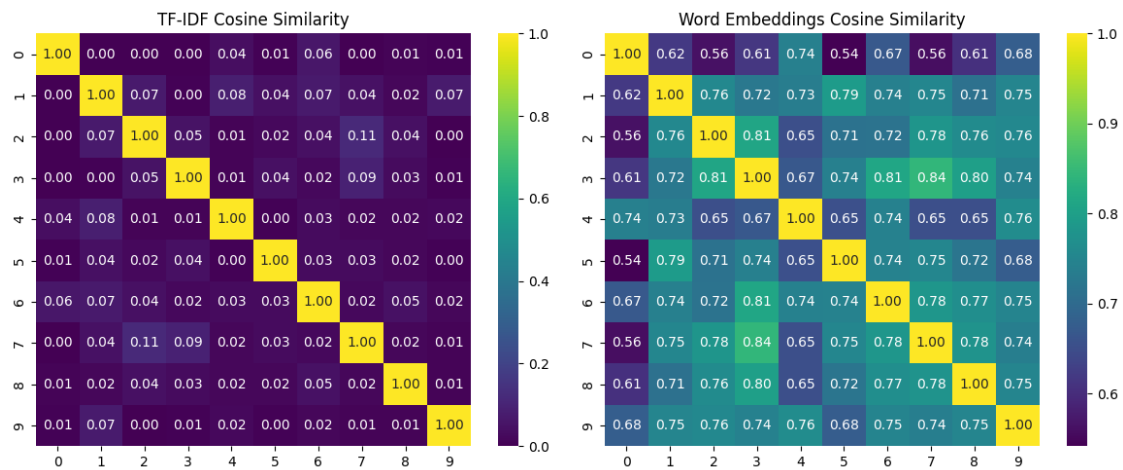
# Plot similarity matrices side by side
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.heatmap(tfidf_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("TF-IDF Cosine Similarity")

plt.subplot(1, 2, 2)
sns.heatmap(w2v_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("Word Embeddings Cosine Similarity")

plt.tight_layout()
plt.show()

```



- [Optional] Represent each item rated by the users using the word vectors from the last layer of BERT. Here, you can directly use the vectors from the pretrained version of BERT available in huggingface. To load the model, install the Transformers library and PyTorch.

```

[58]: import torch
from transformers import BertModel, BertTokenizer

!pip install transformers torch --quiet

# Load model and tokenizer
model_name = "bert-base-uncased"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertModel.from_pretrained(model_name)
model.eval() # set model to evaluation mode

```



```

# Optionally, move model to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

# Function to get BERT representation (using the [CLS] token) for a given text.
def get_bert_vector(text):
    # Handle potential issues with NaN or empty strings
    if not text or text.lower() == "nan":
        return torch.zeros(model.config.hidden_size)
    # Tokenize text with truncation to max_length 512 tokens.
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
↳max_length=512, padding="max_length")
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        outputs = model(**inputs)
    # Use [CLS] token (first token) from the last hidden state as the
↳representation.
    cls_vector = outputs.last_hidden_state[:, 0, :].squeeze().cpu()
    return cls_vector

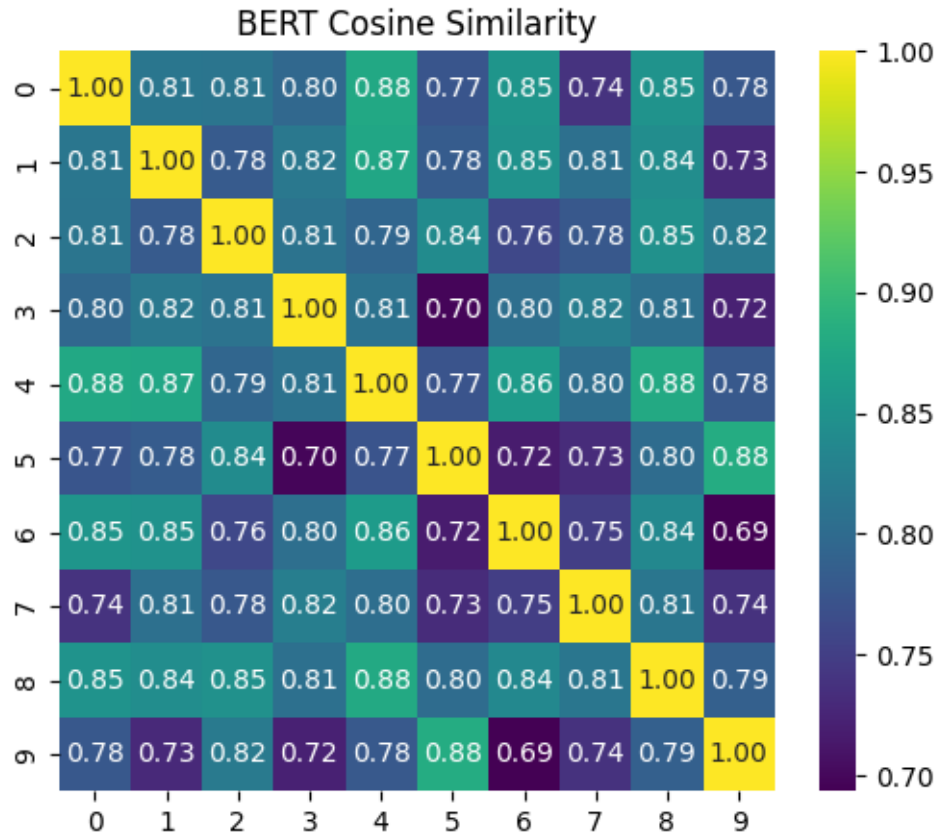
# Compute BERT representations for items in the 'description' column of the
↳subset.
bert_vectors = subset['description'].apply(lambda x: get_bert_vector(str(x)))
print("BERT representations computed for each item.")

# Convert BERT tensors to numpy arrays and stack them
bert_matrix = np.stack(bert_vectors.apply(lambda vec: vec.numpy()).tolist())
bert_sim = cosine_similarity(bert_matrix)

# Plot the cosine similarity matrix for BERT representations
plt.figure(figsize=(6, 5))
sns.heatmap(bert_sim, annot=True, fmt=".2f", cmap="viridis")
plt.title("BERT Cosine Similarity")
plt.show()

```

BERT representations computed for each item.



0.0.4 Week 10 [Mar 3 - Mar 9]: Content-Based Recommender System

In this week, we will continue using the train and test splits defined in Week 6. - Note: similar to the task in Week 9, for this part, you are free to only use a subset of the metadata file, e.g., only using items that are in the train/test splits or using the first n characters/words of the text content. You are also welcome to use additional metadata (e.g., by crawling the internet). Please report and justify your choice.

```
[59]: print("Number of rows:", df_meta.shape[0])
      print("Number of fields:", df_meta.shape[1])
      max_length = df_meta['description'].astype(str).apply(len).max()
      print("Max length of strings in 'description':", max_length)
      print("\nColumns and their data types:")
      print(df_meta.dtypes)
```

Number of rows: 23984

Number of fields: 6

Max length of strings in 'description': 1000

Columns and their data types:

item_id object

```

title                object
description           object
desc_len             int64
preprocessed_description  object
w2v_vector           object
dtype: object

```

- Transform the description column of each item into a TF-IDF representation or other numerical value, e.g., token-count based, that can represent the summaries. Select at least one other factor that can be used as an item feature, for example title. Apply the appropriate preprocessing on the features. These choices should also be reported and justified.

```

[60]: from sklearn.feature_extraction.text import TfidfVectorizer
      from scipy.sparse import hstack
      import numpy as np
      import random
      from tqdm import tqdm
      from sklearn.metrics.pairwise import cosine_similarity

      # Preprocess the title column using the same preprocessing function already
      # defined for descriptions.
      # (Assuming there is a 'title' column in df_meta)
      df_meta['preprocessed_title'] = df_meta['title'].astype(str).apply(lambda x: ' '
      .join(preprocess_text(x)))

      # Build a TF-IDF representation for the preprocessed titles.
      title_tfidf_vectorizer = TfidfVectorizer(tokenizer=lambda x: x.split(),
      lowercase=False)
      tfidf_title_matrix = title_tfidf_vectorizer.
      fit_transform(df_meta['preprocessed_title'])

      # Combine the description TF-IDF matrix (tfidf_matrix, already computed) with
      # the title TF-IDF matrix.
      combined_features = hstack([tfidf_matrix, tfidf_title_matrix])

      # For each item, store its combined feature vector as a new column.
      # Each row is a sparse vector; we extract it as a CSR matrix slice.
      df_meta['combined_features'] = [combined_features.getrow(i) for i in
      range(combined_features.shape[0])]

      print("TF-IDF description vector length:", tfidf_matrix.shape[1])
      print("TF-IDF title vector length:", tfidf_title_matrix.shape[1])
      print("Combined feature vector length:", combined_features.shape[1])

      # Sample 100 random items from the metadata
      random.seed(42) # For reproducibility
      sample_indices = random.sample(range(len(df_meta)), 200)

```

```

sample_items = df_meta.iloc[sample_indices].copy()

# Create a progress bar for the similarity computation
print("Computing pairwise similarities...")
similarities = np.zeros((200, 200))
item_pairs = []

for i in tqdm(range(200)):
    for j in range(i+1, 200):
        # Extract the TF-IDF vectors for the two items
        item_i_vec = sample_items['combined_features'].iloc[i]
        item_j_vec = sample_items['combined_features'].iloc[j]

        # Calculate cosine similarity
        sim = cosine_similarity(item_i_vec, item_j_vec)[0][0]
        similarities[i, j] = sim
        similarities[j, i] = sim # Symmetric matrix

        # Store the item pair and their similarity
        item_pairs.append((i, j, sim))

# Find the most similar pair (highest cosine similarity)
most_similar = max(item_pairs, key=lambda x: x[2])
i_most, j_most, sim_most = most_similar

print(f"\nMost similar items (similarity = {sim_most:.4f}):")
print("\nItem 1:")
print(f"Title: {sample_items['title'].iloc[i_most]}")
print(f>Description excerpt: {sample_items['description'].iloc[i_most][:300]}...
↪")

print("\nItem 2:")
print(f"Title: {sample_items['title'].iloc[j_most]}")
print(f>Description excerpt: {sample_items['description'].iloc[j_most][:300]}...
↪")

# Find the least similar pair (lowest cosine similarity)
least_similar = min(item_pairs, key=lambda x: x[2])
i_least, j_least, sim_least = least_similar

print(f"\nLeast similar items (similarity = {sim_least:.4f}):")
print("\nItem 1:")
print(f"Title: {sample_items['title'].iloc[i_least]}")
print(f>Description excerpt: {sample_items['description'].iloc[i_least][:300]}...
↪.")

print("\nItem 2:")

```

```
print(f"Title: {sample_items['title'].iloc[j_least]}")
print(f"Description excerpt: {sample_items['description'].iloc[j_least][:300]}..
↪.")
```

```
c:\Users\david\anaconda3\envs\WRS\lib\site-
packages\sklearn\feature_extraction\text.py:517: UserWarning: The parameter
'token_pattern' will not be used since 'tokenizer' is not None'
warnings.warn(
```

```
TF-IDF description vector length: 27243
TF-IDF title vector length: 14307
Combined feature vector length: 41550
Computing pairwise similarities...
100%|      | 200/200 [00:15<00:00, 13.27it/s]
```

Most similar items (similarity = 0.9796):

Item 1:

Title: Guitar Chord & Fretboard Note Chart Instructional Easy 11"x17" Poster for Beginners Chords & Notes | A New Song Music |
Description excerpt: nan...

Item 2:

Title: A New Song Music Laminated Guitar Chord & Fretboard Note Chart Instructional Easy Poster for Beginners Chords & Notes 11"x17"
Description excerpt: nan...

Least similar items (similarity = 0.0000):

Item 1:

Title: Guitar Roller String Trees String Tree Guide Retainer Replacement Guitar Parts for Electric Guitar Accessory Set (Chrome .10Pcs)
Description excerpt: Features:.. 100% Brand new and high quality The rollers lessen the stress on your strings which allows the guitar to stay in tune longer. This also prevents wear and tear on the strings which results in longer string life. Base is about 6mm (1/4") wide x 11.5mm (7/16") long Total height including rol...

Item 2:

Title: Hosa HXS-005 REAN XLR3F to 1/4" TRS Pro Balanced Interconnect Cable, 5 Feet
Description excerpt: The Hosa HXS005 Rean Pro 1/4 Inch TRS to XLR3 Female Cable is designed to connect gear with XLR outputs to gear with balanced phone inputs. It is ideal for use in touring and other live-sound applications...

- After you represent each item in a vector space, represent each user in the same vector space.

This can be done by using a simple average of the items the user rated. Note that you can also try to implement better ways to build the user representation.

```
[61]: # Build a mapping from item_id (from metadata) to its combined feature vector
      ↪(a sparse row)
item_profiles = dict(zip(df_meta['item_id'], df_meta['combined_features']))

# Determine the number of features from the first item's vector
num_features = df_meta['combined_features'].iloc[0].toarray().shape[1]

# Create user profiles using only reviews in the training set with ratings >= 1.
user_profiles = {}
for inner_uid, ratings in trainset.ur.items():
    raw_uid = trainset.to_raw_uid(inner_uid)
    # Collect the dense vectors of items rated >= 1 by the user.
    vectors = []
    for inner_iid, rating in ratings:
        if rating >= 1:
            raw_iid = trainset.to_raw_iid(inner_iid)
            if raw_iid in item_profiles:
                # Convert the sparse item vector to a dense array.
                item_vector = item_profiles[raw_iid].toarray().flatten()
                vectors.append(item_vector)
            # Compute the average of the selected vectors, or initialize with a zero
            ↪vector if none.
        if vectors:
            user_profile = np.mean(vectors, axis=0)
        else:
            user_profile = np.zeros(num_features)
    user_profiles[raw_uid] = user_profile

print("Created user profiles for {} users using only items with ratings >= 1.".
      ↪format(len(user_profiles)))
```

Created user profiles for 800 users using only items with ratings >= 1.

- Calculate the user-item rating prediction for an item by using a similarity/distance metric between the user and the item representations. A metric such as cosine similarity or Euclidean distance can be used. Motivate your choice.

```
[62]: import random
      from sklearn.metrics import mean_squared_error

      def predict_rating(user_id, item_id, min_rating=1, max_rating=5):
          # Retrieve the user profile vector and item feature vector.
          user_vector = user_profiles.get(user_id)
          if user_vector is None:
              print("Unknown user.")
```

```

        return None
    if item_id not in item_profiles:
        print("Unknown item.")
        return None

    # Convert the sparse item vector to a dense representation.
    item_vector = item_profiles[item_id].toarray().flatten()

    # Do NOT mean-center the item vector, since our user profiles are built
    ↪from raw item vectors.
    # Reshape vectors to 2D arrays for cosine_similarity.
    user_vector_2d = user_vector.reshape(1, -1)
    item_vector_2d = item_vector.reshape(1, -1)

    # Compute cosine similarity between the user and item vectors.
    similarity = cosine_similarity(user_vector_2d, item_vector_2d)[0][0]

    # If your cosine similarities are in [0, 1] (common with non-negative
    ↪vectors),
    # you may want to map directly as follows:
    predicted_rating = similarity * (max_rating - min_rating) + min_rating
    #
    # Otherwise, if similarities can span [-1, 1], you can use:
    # predicted_rating = (similarity + 1) * (max_rating - min_rating) / 2 +
    ↪min_rating
    return predicted_rating

# Sample 100 random (user_id, item_id, rating) tuples from the test set.
sample_df = test_df.sample(n=100, random_state=42)

true_ratings = []
predicted_ratings_model = []    # Using predict_rating (content-based)
predicted_ratings_random = []  # Random predictor (uniformly random 1 to 5)
predicted_ratings_constant = [] # Constant predictor (always predict 3)

for _, row in sample_df.iterrows():
    user_id = row['user_id']
    item_id = row['item_id']
    true_rating = row['rating']

    # Content-based predictor from our model.
    pred = predict_rating(user_id, item_id)
    if pred is not None:
        true_ratings.append(true_rating)
        predicted_ratings_model.append(pred)
    else:
        print(f"Prediction not available for user {user_id} and item {item_id}")

```

```

# Random predictor: generate a random float rating between 1 and 5.
random_rating = random.uniform(1, 5)
predicted_ratings_random.append(random_rating)

# Constant predictor: always predict 3.
predicted_ratings_constant.append(3)

# Compute RMSE for each predictor.
mse_model = mean_squared_error(true_ratings, predicted_ratings_model)
rmse_model = mse_model ** 0.5

mse_random = mean_squared_error(true_ratings, predicted_ratings_random)
rmse_random = mse_random ** 0.5

mse_constant = mean_squared_error(true_ratings, predicted_ratings_constant)
rmse_constant = mse_constant ** 0.5

print("RMSE for content-based predictor:", rmse_model)
print("RMSE for random predictor (ratings between 1-5):", rmse_random)
print("RMSE for constant predictor (always predict 3):", rmse_constant)

```

```

RMSE for content-based predictor: 3.1830293882582072
RMSE for random predictor (ratings between 1-5): 2.178067695909589
RMSE for constant predictor (always predict 3): 1.7378147196982767

```

- Report Precision@10, MAP@10, MRR@10, hit rate and coverage using ratings 4 in the test set. Compare the results with the models from previous weeks.

```

[63]: from numpy.linalg import norm
import numpy as np
from tqdm import tqdm
import random

# Precompute each item's raw vector and its norm (without subtracting the
    ↪ global mean).
item_vectors = {}
item_norms = {}
for item in item_profiles:
    vec = item_profiles[item].toarray().flatten() # use raw vector
    item_vectors[item] = vec
    item_norms[item] = norm(vec)

# Use the top 20 users with the most reviews in the train set.
#sample_users = sorted(user_rating_counts, key=user_rating_counts.get,
    ↪ reverse=True)[:20]

sample_users = random.sample(list(user_rating_counts.keys()), 100)

```



```

recs_sample = {}
all_items = set(item_profiles.keys())

# Process each user.
for user in tqdm(sample_users, desc="Processing sample_df users"):
    try:
        inner_uid = trainset.to_inner_uid(user)
        rated_items = {trainset.to_raw_iid(inner_iid) for inner_iid, _ in
            ↪trainset.ur[inner_uid]}
        except Exception as e:
            rated_items = set() # Assume no rated items if user not in training
            ↪set.

        unobserved_items = list(all_items - rated_items)
        predictions = []
        user_vec = user_profiles[user]
        user_norm = norm(user_vec)

        for item in unobserved_items:
            vec = item_vectors[item]
            item_n = item_norms[item]
            if user_norm == 0 or item_n == 0:
                sim = 0
            else:
                sim = np.dot(user_vec, vec) / (user_norm * item_n)
            predictions.append((item, sim))

        predictions.sort(key=lambda x: x[1], reverse=True)
        top_10 = predictions[:10] # List of (item, sim) pairs
        recs_sample[user] = top_10

# For evaluation, extract only the recommended item IDs.
recs_sample_eval = {user: [item for item, sim in recs_sample[user]] for user in
    ↪recs_sample}

# Filter ground_truth to only include users in sample_users.
filtered_ground_truth = {user: ground_truth[user] for user in sample_users if
    ↪user in ground_truth}

# Evaluate the recommendations using the provided evaluate function.
results = evaluate(recs_sample_eval, filtered_ground_truth, k=10,
    ↪total_items=len(item_profiles))
print("Evaluation metrics for sample_df users (Hit Rate, Precision@10, MAP@10,
    ↪MRR@10, Coverage):", results)

```

```
# Print for each user the 10 recommended items and their cosine similarity
↪scores.
print("\nRecommended items with cosine similarity for each user:")
for user in recs_sample:
    print(f"\nUser: {user}")
    for item, sim in recs_sample[user]:
        print(f"    {item}: cosine similarity = {sim:.4f}")
```

Processing sample_df users: 100%| | 100/100 [05:32<00:00, 3.33s/it]

Evaluation metrics for sample_df users (Hit Rate, Precision@10, MAP@10, MRR@10, Coverage): (0.02040816326530612, 0.0020408163265306124, 0.00040816326530612246, 0.0020408163265306124, 0.013717478318879253)

Recommended items with cosine similarity for each user:

User: AEPNDOJU3WRZU4C4BPTUTJTA74TQ

B07PDZ4KKS: cosine similarity = 0.5514
 B08PP67XCM: cosine similarity = 0.5340
 B00USQYOCK: cosine similarity = 0.5318
 B019RJJWI0: cosine similarity = 0.5296
 B07NPDHXKD: cosine similarity = 0.5250
 B01M6COY4T: cosine similarity = 0.5225
 B07DSY3GGH: cosine similarity = 0.5212
 B09M42KWW2: cosine similarity = 0.5175
 B077RGVHRJ: cosine similarity = 0.5165
 B01LABAI1I: cosine similarity = 0.5164

User: AE7GVDNRY5NABEP253Y3GMDUZOZA

B005SUT9I6: cosine similarity = 0.4967
 B005SUT9UY: cosine similarity = 0.4901
 B0002E2XCW: cosine similarity = 0.4888
 B0987C2TBZ: cosine similarity = 0.4888
 B0002E2S9A: cosine similarity = 0.4876
 B009J8DXQW: cosine similarity = 0.4733
 B0002D0CFS: cosine similarity = 0.4532
 B07MCD7S3G: cosine similarity = 0.4469
 B016PHMM34: cosine similarity = 0.4469
 BOBYJN7JWQ: cosine similarity = 0.4402

User: AHVGICSVXM66E4SXX23PH7R3Q74Q

B018SA2CPE: cosine similarity = 0.6947
 B08PP67XCM: cosine similarity = 0.6925
 B01LABAI1I: cosine similarity = 0.6802
 B09M42KWW2: cosine similarity = 0.6801
 B077RGVHRJ: cosine similarity = 0.6767
 B07HQWXHJW: cosine similarity = 0.6744
 B01M6COY4T: cosine similarity = 0.6744

B06XY6MPYT: cosine similarity = 0.6714
B07FW4BW45: cosine similarity = 0.6608
B07VMR6TVT: cosine similarity = 0.6541

User: AF6K6UZP2WMQIEH76FCAHEQBCSGQ

B08GS84HXJ: cosine similarity = 0.5848
B016Z435Q0: cosine similarity = 0.5759
B07KVT1DMQ: cosine similarity = 0.5738
B07KVT68WY: cosine similarity = 0.5738
B093WDHWW1: cosine similarity = 0.5658
B07W53561Q: cosine similarity = 0.5606
B08JZ3BSZH: cosine similarity = 0.5587
B097SRM7BZ: cosine similarity = 0.5578
B08KXXRFGP: cosine similarity = 0.5573
B0B1ZD5SXF: cosine similarity = 0.5553

User: AHX2B4DEER2QR3IU3CCNB3CWC6TA

B09BR11LYJ: cosine similarity = 0.4648
B09BYVPFTT: cosine similarity = 0.4648
B0928SKWF5: cosine similarity = 0.4641
B086YBWG53: cosine similarity = 0.4525
B08B1LZDDT: cosine similarity = 0.4504
B08GM1RLCT: cosine similarity = 0.4504
B0875W74D8: cosine similarity = 0.4502
B0C899312H: cosine similarity = 0.4428
B08R3FDHV9: cosine similarity = 0.4421
B0C542XXJ8: cosine similarity = 0.4415

User: AGMTCMP27WCMXZ3R3FJ5F4KDABA

B0002DOL4K: cosine similarity = 0.3175
B0002GYX50: cosine similarity = 0.3073
B0002GYX78: cosine similarity = 0.3073
B09G5PFGV6: cosine similarity = 0.2954
B01C5TBYYY: cosine similarity = 0.2929
B07SGPV7QM: cosine similarity = 0.2837
B000P1RB9C: cosine similarity = 0.2823
B01M708ZUT: cosine similarity = 0.2814
B01MG3QVCK: cosine similarity = 0.2774
B0868L55QS: cosine similarity = 0.2755

User: AEPN2RD5DNLVBU40QPDDZLM5EAMA

B0B8DR9RGD: cosine similarity = 0.4315
B018K9JZW6: cosine similarity = 0.4300
B0B8LK7MTH: cosine similarity = 0.4242
B098WXZ874: cosine similarity = 0.4045
B089CKM49N: cosine similarity = 0.3754
B000B6AGAC: cosine similarity = 0.3733
B07BWS3JHL: cosine similarity = 0.3688

B0002E2S9A: cosine similarity = 0.3688
B0BKRBZ29M: cosine similarity = 0.3655
B007MY5BII: cosine similarity = 0.3583

User: AEUQPJ2W0GSXVF6S37NWLBDIAR5A

B08GS84HXJ: cosine similarity = 0.5312
B07RXYP59L: cosine similarity = 0.5249
B016Z435Q0: cosine similarity = 0.5215
B0871QL7ZR: cosine similarity = 0.5214
B07KVT1DMQ: cosine similarity = 0.5191
B07KVT68WY: cosine similarity = 0.5191
B01CSAATY6: cosine similarity = 0.5136
B08L76JQBM: cosine similarity = 0.5132
B097SRM7BZ: cosine similarity = 0.5088
B08P2LW2MP: cosine similarity = 0.5056

User: AG3PQBC2VIMREI2WXU7IFYRIGTPQ

B09PRKZ6FN: cosine similarity = 0.6365
B07Q8F4S4J: cosine similarity = 0.6145
B093YCH6LB: cosine similarity = 0.6092
B09NL4ZR9L: cosine similarity = 0.6040
B086YBWG53: cosine similarity = 0.6032
B09BYVPFTT: cosine similarity = 0.5992
B08PZ5FQ3H: cosine similarity = 0.5954
B0BGPPJQ97: cosine similarity = 0.5939
B08N9ZNV8F: cosine similarity = 0.5922
B0928SKWF5: cosine similarity = 0.5908

User: AG7EI3UOXIQXEL5QDZPVKPSVR4UQ

B07YMPWC71: cosine similarity = 0.3400
B0064BD6SW: cosine similarity = 0.3307
B003IEQ63K: cosine similarity = 0.3072
B0764R1F7Q: cosine similarity = 0.2976
B00IXXCT9G: cosine similarity = 0.2936
B0B8P1YW7L: cosine similarity = 0.2934
B072HH4RWT: cosine similarity = 0.2917
B00TVFYE2U: cosine similarity = 0.2917
B071LMDNHY: cosine similarity = 0.2865
B07Q9JB3CF: cosine similarity = 0.2851

User: AGIOSDBFWZCPIQVGUKQLS6WFRQ3A

B073W8P6QY: cosine similarity = 0.6882
B09PRKZ6FN: cosine similarity = 0.6686
B0769JX3RR: cosine similarity = 0.6632
B093WDHWW1: cosine similarity = 0.6632
B00WHLLDWO: cosine similarity = 0.6598
B07RK6FQ9J: cosine similarity = 0.6590
B01CSAATY6: cosine similarity = 0.6511

B07MCGRLMT: cosine similarity = 0.6510
B07Y1FSYYB: cosine similarity = 0.6503
B07RXYP59L: cosine similarity = 0.6485

User: AG6QMKE26TOI4SJCPJD2F3APWE7Q

B079QDMQHF: cosine similarity = 0.4969
B07BLG25CM: cosine similarity = 0.4921
B09JV2KKJP: cosine similarity = 0.4778
B09T7P2RNC: cosine similarity = 0.4641
B0BFXNCQRJ: cosine similarity = 0.4481
B079QDGK51: cosine similarity = 0.4344
B09BC1S3JS: cosine similarity = 0.4117
B07BLBQFQ6: cosine similarity = 0.3973
B0B8DR9RGD: cosine similarity = 0.3933
B0BKZ7PZ1X: cosine similarity = 0.3849

User: AHETPYRS55HBETNZJ7ACESF5QNCQ

B07M81J38F: cosine similarity = 0.4735
B01LEZLR9M: cosine similarity = 0.4670
B0BJY2MYD1: cosine similarity = 0.4562
B07DWMCT3X: cosine similarity = 0.4533
B07B4XLP57: cosine similarity = 0.4429
B000K6CS40: cosine similarity = 0.4406
B09BF8XDF4: cosine similarity = 0.4388
B07DWY7R2X: cosine similarity = 0.4354
B0BSBVQBVR: cosine similarity = 0.4334
B00162ZR0K: cosine similarity = 0.4334

User: AHBD4JG5WW5UH6YRVPW25FKQIFYA

B0C3H7SNPQ: cosine similarity = 0.3185
B0048UCGG0: cosine similarity = 0.3087
B07217RS84: cosine similarity = 0.2917
B0BCXSJL9Y: cosine similarity = 0.2911
B09G5XH5YD: cosine similarity = 0.2899
B08VJBN9S7: cosine similarity = 0.2837
B0BCXTPJR2: cosine similarity = 0.2833
B0B819R6CN: cosine similarity = 0.2762
B0BCXS7X26: cosine similarity = 0.2761
B0BXT1HVWC: cosine similarity = 0.2757

User: AEHWHU26Z2VEZQY3IEEW5HBIGX5Q

B00B1ZONFG: cosine similarity = 0.3575
B09BF8XDF4: cosine similarity = 0.3270
B079W7NWNH: cosine similarity = 0.3206
B000K6CS40: cosine similarity = 0.3184
B07V66BWL7: cosine similarity = 0.3183
B0BKZ5F8BS: cosine similarity = 0.3121
B07CQ919R3: cosine similarity = 0.3101

B06XB3FQKB: cosine similarity = 0.3093
B07B4XLP57: cosine similarity = 0.3090
B07DWMCT3X: cosine similarity = 0.3061

User: AF3NTN5FD5C3WWBBL4EYNZPA7GVA

B09NVYW22T: cosine similarity = 0.5102
B07RP68CXC: cosine similarity = 0.5091
B0B1ZD5SXF: cosine similarity = 0.5085
B09Q8V58BV: cosine similarity = 0.4913
B09J24N1WJ: cosine similarity = 0.4879
B06XFFMDKK: cosine similarity = 0.4867
B07SDRG1G9: cosine similarity = 0.4833
B07FBD8F7C: cosine similarity = 0.4797
B08P4G7S11: cosine similarity = 0.4778
B088LNMGHZ: cosine similarity = 0.4746

User: AF7L3ET5M007VTDNU05QHQUESIM7Q

B01N7KFOT2: cosine similarity = 0.5528
B07MDLZ8HC: cosine similarity = 0.5337
B00CS0JMU4: cosine similarity = 0.5223
B01N9JVOWU: cosine similarity = 0.5178
B01MT1NUDY: cosine similarity = 0.5171
B07C779ZYP: cosine similarity = 0.5166
B01MDTSGDT: cosine similarity = 0.5135
B00GEDQAJW: cosine similarity = 0.5130
B00GEDQAY2: cosine similarity = 0.5130
B01L8C10W0: cosine similarity = 0.5089

User: AG6EDKW4LTUP3EKVJUFQWJTF3AYA

B0BHL53RKC: cosine similarity = 0.4365
B0BZHZNSTN: cosine similarity = 0.4163
B06XFFMDKK: cosine similarity = 0.4132
B00LMOVXFO: cosine similarity = 0.4012
B0B8DR9RGD: cosine similarity = 0.4004
B07F75YNGD: cosine similarity = 0.3988
B01M7QOF1E: cosine similarity = 0.3969
B081T2PZHW: cosine similarity = 0.3965
B0BH3GJ3ML: cosine similarity = 0.3961
B08DV1LX3J: cosine similarity = 0.3958

User: AFQ7VJBUAFMCD6WLRMNJVL5IZGUA

B004FRHKA2: cosine similarity = 0.5905
B007VOGU80: cosine similarity = 0.5893
B005LYIW3W: cosine similarity = 0.5484
B005MOCK9M: cosine similarity = 0.5309
B004FEH790: cosine similarity = 0.5196
B005MOMUQK: cosine similarity = 0.4995
B09NVYW22T: cosine similarity = 0.4605

B006XXT8EW: cosine similarity = 0.4533
B00K8MZWI: cosine similarity = 0.4522
B00FGLJPXQ: cosine similarity = 0.4522

User: AF3RJBAVKA5SURCOISFLDBHF6UOA

B0BHL53RKC: cosine similarity = 0.5448
B0BLSL5D2R: cosine similarity = 0.5313
B0BZHNTSN: cosine similarity = 0.5241
B07TQ8Z1SG: cosine similarity = 0.5210
B081T2PZHW: cosine similarity = 0.5191
B08DFQ6SXB: cosine similarity = 0.5116
B07VJJ3T1N: cosine similarity = 0.5105
B07HH697NZ: cosine similarity = 0.5094
B017P7Z2QC: cosine similarity = 0.5091
B0B4BB58CX: cosine similarity = 0.5080

User: AFZ4XEURSL7VMVPEKG5SHU30AULQ

B0064RN6UY: cosine similarity = 0.4320
B006XY8N9M: cosine similarity = 0.4211
B00UCUEOG4: cosine similarity = 0.3945
B0064RVIKY: cosine similarity = 0.3935
B01K7J1ECG: cosine similarity = 0.3881
B01K7J1H8M: cosine similarity = 0.3864
B0BFKQ9QXD: cosine similarity = 0.3806
B0002CZVS2: cosine similarity = 0.3791
B0BG64YHQY: cosine similarity = 0.3750
B0BFKPM8MQ: cosine similarity = 0.3746

User: AHW2D5AFKMWOIVCEXUZJSB43TAHQ

B09PRKZ6FN: cosine similarity = 0.2998
B06XFFMDKK: cosine similarity = 0.2933
B0BFR5LN7M: cosine similarity = 0.2889
B0C542XXJ8: cosine similarity = 0.2837
B08R3FDHV9: cosine similarity = 0.2829
B08N9ZNV8F: cosine similarity = 0.2815
B08DV1LX3J: cosine similarity = 0.2814
B09M7P8SBH: cosine similarity = 0.2794
B07TLJ3KSB: cosine similarity = 0.2788
B086XDX71B: cosine similarity = 0.2774

User: AGOJCISATDQUG3HCXSIFAPT7JMOA

B079L5G8FK: cosine similarity = 0.5630
B07WRWDJHV: cosine similarity = 0.5603
B017P7Z2QC: cosine similarity = 0.5518
B08TX3TRZD: cosine similarity = 0.5505
B07TQ8Z1SG: cosine similarity = 0.5474
B07VJJ3T1N: cosine similarity = 0.5385
B08R8YPHRH: cosine similarity = 0.5383

B00RUW65WS: cosine similarity = 0.5356
B07SHMH8WK: cosine similarity = 0.5338
B07DVX841V: cosine similarity = 0.5312

User: AHLRDVX6R0CCPNXTCZLY2KPVLCHA

B083BTY7FK: cosine similarity = 0.6148
B083BG1416: cosine similarity = 0.6063
B093WDHWW1: cosine similarity = 0.5976
B07BK51JHB: cosine similarity = 0.5972
B0769JX3RR: cosine similarity = 0.5922
B0B176CWVM: cosine similarity = 0.5849
B07C7BKVL8: cosine similarity = 0.5846
B07RK6FQ9J: cosine similarity = 0.5820
B07RYQ1X42: cosine similarity = 0.5819
B09PRKZ6FN: cosine similarity = 0.5808

User: AHLM3EPKW4KDTZWLD5IAMHOCWUKQ

B0B8DR9RGD: cosine similarity = 0.4320
B09D971MJN: cosine similarity = 0.4007
B0B9M4SJ24: cosine similarity = 0.3979
B079QDMQHF: cosine similarity = 0.3923
B0B8LQFN3C: cosine similarity = 0.3903
B07M81J38F: cosine similarity = 0.3895
B09MSCVKP8: cosine similarity = 0.3863
B07B16JL73: cosine similarity = 0.3851
B00XTADXNE: cosine similarity = 0.3847
B00E9SXE7A: cosine similarity = 0.3810

User: AHZJVJET7N5JKBRKL6E7SMZV6FKQ

B00CJ3AGU0: cosine similarity = 0.2976
B01B1JHEX4: cosine similarity = 0.2868
B01448UA08: cosine similarity = 0.2860
B01ANNKMI: cosine similarity = 0.2805
B004FRHKA2: cosine similarity = 0.2803
B007VOGU80: cosine similarity = 0.2786
B005LYIW3W: cosine similarity = 0.2693
B004UOE3G4: cosine similarity = 0.2685
B08PFF2Y92: cosine similarity = 0.2653
B004FEH790: cosine similarity = 0.2564

User: AFN6UNROF06Z5BDNTMZR2DK6QNKQ

B07MVLVHTS: cosine similarity = 0.3942
B09DDQ3T57: cosine similarity = 0.3925
B09BF8XDF4: cosine similarity = 0.3881
B07DWMCT3X: cosine similarity = 0.3815
B07DWY7R2X: cosine similarity = 0.3794
B07DWLYGKH: cosine similarity = 0.3701
B07DWY44X8: cosine similarity = 0.3608

B0002E2NCM: cosine similarity = 0.3425
B07C5DLBZT: cosine similarity = 0.3407
B0B2M1GPR7: cosine similarity = 0.3405

User: AEQJ4C6IRPWITBGZ3PVJJB6DLW3Q

B00PU06838: cosine similarity = 0.4212
B07KCPBC98: cosine similarity = 0.3617
B0041XTHV0: cosine similarity = 0.3500
B07WYNVBQG: cosine similarity = 0.3443
B09Y2HK4M7: cosine similarity = 0.3434
B07VJJ3T1N: cosine similarity = 0.3411
B0774G3FNJ: cosine similarity = 0.3406
B01D2QAKJY: cosine similarity = 0.3405
B07DC2FFKS: cosine similarity = 0.3389
B07VWY1P8H: cosine similarity = 0.3377

User: AE5VPKX64CVOPH6X75C7EQXAIGLA

B07MF2HPQV: cosine similarity = 0.4208
B06XJDTC2N: cosine similarity = 0.4206
B08N9ZNV8F: cosine similarity = 0.4186
B09BYVPFTT: cosine similarity = 0.4174
B08B1LZDDT: cosine similarity = 0.4170
B08GM1RLCT: cosine similarity = 0.4170
B00MN2FBEL: cosine similarity = 0.4166
B07MTX32J4: cosine similarity = 0.4161
B08R3FDHV9: cosine similarity = 0.4152
B017P7Z2QC: cosine similarity = 0.4130

User: AFN4MT6SZPZWDQ5D5AGJ6WP3E3YQ

B01ANNKMI: cosine similarity = 0.3996
B004FRHKA2: cosine similarity = 0.3991
B005MOMUQK: cosine similarity = 0.3984
B007VOGU80: cosine similarity = 0.3963
B005LYIW3W: cosine similarity = 0.3803
B005MOCK9M: cosine similarity = 0.3781
B07SDRG1G9: cosine similarity = 0.3714
B004FEH790: cosine similarity = 0.3660
B004FEBS8K: cosine similarity = 0.3565
B00K8MZWI: cosine similarity = 0.3556

User: AH5L2ZWXT6BSAUYQCH52NWFD6BVA

B09PRKZ6FN: cosine similarity = 0.5503
B0BGPPJQ97: cosine similarity = 0.5206
B099NFZP4S: cosine similarity = 0.5195
B0BH599FJY: cosine similarity = 0.5117
B093YCH6LB: cosine similarity = 0.5063
B096YL3M8Q: cosine similarity = 0.5063
B07TQSDF9D: cosine similarity = 0.5050

B08JLWKYXP: cosine similarity = 0.5050
B093WDHWW1: cosine similarity = 0.5043
B07C44MK1N: cosine similarity = 0.5043

User: AES7KU7JY6KI2MNQ4N4IP2YCMTZQ

B099NFZP4S: cosine similarity = 0.5589
B09NL4ZR9L: cosine similarity = 0.5363
B09PRKZ6FN: cosine similarity = 0.5354
B07Q8F4S4J: cosine similarity = 0.5301
B0BGPPJQ97: cosine similarity = 0.5271
B0951W3WCF: cosine similarity = 0.5265
B093YCH6LB: cosine similarity = 0.5168
B07SDRG1G9: cosine similarity = 0.5135
B096YL3M8Q: cosine similarity = 0.5078
B08MLD6M1W: cosine similarity = 0.5053

User: AFZJSGJLARXCYV02NTD6IE63E2HQ

B07JQDV6R3: cosine similarity = 0.6500
B085FX6R4V: cosine similarity = 0.6500
B07R7GFFX4: cosine similarity = 0.6445
B08HV3P63M: cosine similarity = 0.6415
B0749L8YT8: cosine similarity = 0.6320
B09TH7DXB6: cosine similarity = 0.6247
B07WS5NRQF: cosine similarity = 0.6237
B07VJJ3T1N: cosine similarity = 0.6222
B0B14V1FMK: cosine similarity = 0.6220
B07T9DKXPD: cosine similarity = 0.6187

User: AGJ7RLVCMTUBOVBIHLK2TXPNJL3A

B0928SKWF5: cosine similarity = 0.6275
B09BYVPFTT: cosine similarity = 0.6229
B09PRKZ6FN: cosine similarity = 0.6131
B09BR11LYJ: cosine similarity = 0.6129
B09K81GB8V: cosine similarity = 0.6101
B08B1LZDDT: cosine similarity = 0.6076
B08GM1RLCT: cosine similarity = 0.6076
B093YCH6LB: cosine similarity = 0.6064
B09L4NLRGJ: cosine similarity = 0.6031
B0875W74D8: cosine similarity = 0.6001

User: AFTSNLQBX4R6BYTB337XHJIYS4SQ

B09G5PFGV6: cosine similarity = 0.4260
B0B7C52N2F: cosine similarity = 0.4165
B0064RTSUQ: cosine similarity = 0.3901
B00IXXCT9G: cosine similarity = 0.3888
B09XSC5SMJ: cosine similarity = 0.3776
B08GKXLHY1: cosine similarity = 0.3753
B09G5HY43T: cosine similarity = 0.3723

B08KH7WCQ8: cosine similarity = 0.3674
B00EL7666K: cosine similarity = 0.3658
B0064RTSOG: cosine similarity = 0.3617

User: AGUG7ONCKL4JOYS6AILUT3AGOCWQ

B093WDHWW1: cosine similarity = 0.5825
B09QMCFCZY: cosine similarity = 0.5806
B083BG1416: cosine similarity = 0.5782
B004FRHKA2: cosine similarity = 0.5769
B09XMQ92RM: cosine similarity = 0.5743
B09PRKZ6FN: cosine similarity = 0.5709
B08LVQKMKZ: cosine similarity = 0.5704
B07RK6FQ9J: cosine similarity = 0.5704
B09Q8V58BV: cosine similarity = 0.5694
B0769JX3RR: cosine similarity = 0.5681

User: AG23AW56WZ7KPYYAS4A3YUDTYWCQ

B09ZTWRRPR: cosine similarity = 0.3878
B0BHQ6TF9Q: cosine similarity = 0.3817
B07WYNVBQG: cosine similarity = 0.3807
B07VJJ3T1N: cosine similarity = 0.3801
B09BR11LYJ: cosine similarity = 0.3796
B09CPZ2QS9: cosine similarity = 0.3790
B09L4NLRGJ: cosine similarity = 0.3776
B07VS1Y4KL: cosine similarity = 0.3747
B06XFFMDKK: cosine similarity = 0.3731
B0928SKWF5: cosine similarity = 0.3691

User: AHELQMGQBLWIV23NL2JN506B55ZA

B0BMZHPKTH: cosine similarity = 0.3498
B0BYJN7JWQ: cosine similarity = 0.3463
B0002D0CFS: cosine similarity = 0.3438
B0002D0CEO: cosine similarity = 0.3367
B0002GX7XY: cosine similarity = 0.3331
B0064BCRQE: cosine similarity = 0.3313
B0BR2N43GC: cosine similarity = 0.3285
B0B81K2Y3H: cosine similarity = 0.3275
B09R6KV6QX: cosine similarity = 0.3183
B003B085BC: cosine similarity = 0.3044

User: AFFOT7AORQAWKU6KKPDHR3FOA5AA

B0BLLF6ZCS: cosine similarity = 0.5085
B0B8Z1M2S1: cosine similarity = 0.5085
B09FSRVQ61: cosine similarity = 0.5085
B07QNR8X61: cosine similarity = 0.4674
B07QQZDLP8: cosine similarity = 0.4100
B0032TN7DS: cosine similarity = 0.3867
B002NLSCPE: cosine similarity = 0.3750

B00721D2ZU: cosine similarity = 0.3658
B07MCD7S3G: cosine similarity = 0.3487
B016PHMM34: cosine similarity = 0.3487

User: AHZQPH7HHSWLUIQFWEQ54NNKKN6A

BOB8DR9RGD: cosine similarity = 0.4057
BOB8LQFN3C: cosine similarity = 0.4029
BOB96H2CHT: cosine similarity = 0.3972
BOB8LK7MTH: cosine similarity = 0.3935
B098WXZ874: cosine similarity = 0.3814
B09MSCVKP8: cosine similarity = 0.3739
B07JW4Y9YS: cosine similarity = 0.3674
BOBKRBZ29M: cosine similarity = 0.3648
B018K9JZW6: cosine similarity = 0.3590
B07C93Z5XZ: cosine similarity = 0.3567

User: AHR6CSQFJVG5AHI25CP4NTJDEZFQ

B06XFFMDKK: cosine similarity = 0.3942
B0928SKWF5: cosine similarity = 0.3942
B09BYVPFTT: cosine similarity = 0.3898
B08B1LZDDT: cosine similarity = 0.3790
B08GM1RLCT: cosine similarity = 0.3790
B099NFZP4S: cosine similarity = 0.3770
B08N9ZNV8F: cosine similarity = 0.3713
B09Q5KGRV3: cosine similarity = 0.3701
B09BR11LYJ: cosine similarity = 0.3700
B09PRKZ6FN: cosine similarity = 0.3691

User: AGUSWVPMRRKPI5CS56UWY4VXFZJA

BOB8DR9RGD: cosine similarity = 0.3895
B07M81J38F: cosine similarity = 0.3874
B0002E2XCW: cosine similarity = 0.3807
B0987C2TBZ: cosine similarity = 0.3807
B0002E2S9A: cosine similarity = 0.3789
B005SUT9I6: cosine similarity = 0.3783
B009J8DXQW: cosine similarity = 0.3732
B005SUT9UY: cosine similarity = 0.3623
B00MKNL9P0: cosine similarity = 0.3574
B016K1890U: cosine similarity = 0.3564

User: AENRJ7FMJYCBVBZPTFSNDRRGX4WQ

B06XFFMDKK: cosine similarity = 0.5325
B08DV1LX3J: cosine similarity = 0.5270
BOBHL53RKC: cosine similarity = 0.5244
B01M7QOF1E: cosine similarity = 0.5116
BOB14V1FMK: cosine similarity = 0.5105
B07F75YNGD: cosine similarity = 0.5098
B077SZ667X: cosine similarity = 0.5043

B0BHQ7T6FZ: cosine similarity = 0.5013
B01M6COY4T: cosine similarity = 0.4965
B08PP67XCM: cosine similarity = 0.4957

User: AE57MRF2R2ALCC6H5WQLFKT7KSSA

B06XFFMDKK: cosine similarity = 0.4262
B08DV1LX3J: cosine similarity = 0.4202
B09BYVPFTT: cosine similarity = 0.4194
B08R3FDHV9: cosine similarity = 0.4186
B0C899312H: cosine similarity = 0.4136
B08TX3TRZD: cosine similarity = 0.4136
B098LHYD3K: cosine similarity = 0.4078
B01M7QOF1E: cosine similarity = 0.4052
B0BZHNTSN: cosine similarity = 0.4041
B09L4NLRGJ: cosine similarity = 0.4017

User: AHESWMVRGAHKN2L3QVZCA5MJHTMA

B01DE4DTAG: cosine similarity = 0.5379
B08B1LZDDT: cosine similarity = 0.5112
B08GM1RLCT: cosine similarity = 0.5112
B08R3FDHV9: cosine similarity = 0.5060
B083JJD715: cosine similarity = 0.5007
B07WS2BBVR: cosine similarity = 0.4979
B006VR66RW: cosine similarity = 0.4929
B099NFZP4S: cosine similarity = 0.4911
B093YCH6LB: cosine similarity = 0.4899
B09BYVPFTT: cosine similarity = 0.4891

User: AEP6VSEK3Y5MHGJCG2RGHM2EHCLA

B00JL5I57U: cosine similarity = 0.4007
B0064BD6SW: cosine similarity = 0.3896
B071LMDNHY: cosine similarity = 0.3631
B09TGD5X27: cosine similarity = 0.3578
B01H4D4MBS: cosine similarity = 0.3544
B072HH4RWT: cosine similarity = 0.3390
B00TVFYE2U: cosine similarity = 0.3390
B0BKZ5F8BS: cosine similarity = 0.3386
B0178ZWFE: cosine similarity = 0.3250
B09S2VB3KW: cosine similarity = 0.3227

User: AFFELFYTX2FJCTZLASZLK3K7FQNA

B06XFFMDKK: cosine similarity = 0.5978
B08DV1LX3J: cosine similarity = 0.5783
B07VJJ3T1N: cosine similarity = 0.5657
B0B4BB58CX: cosine similarity = 0.5551
B07MCGWPKF: cosine similarity = 0.5526
B09BYVPFTT: cosine similarity = 0.5479
B0BVYDCP5X: cosine similarity = 0.5465

B08P8JCQYJ: cosine similarity = 0.5461
B07W5S2RP2: cosine similarity = 0.5461
B07FM97YFY: cosine similarity = 0.5459

User: AGJNGLJSUQGXIWIXDARX26DQYFQQ

B00JLPHKDA: cosine similarity = 0.4262
B09BR11LYJ: cosine similarity = 0.4182
B0928SKWF5: cosine similarity = 0.4081
B06XFFMDKK: cosine similarity = 0.4048
B09BYVPFTT: cosine similarity = 0.3994
B088D2WS29: cosine similarity = 0.3965
B08PZ5FQ3H: cosine similarity = 0.3965
B08B1LZDDT: cosine similarity = 0.3958
B08GM1RLCT: cosine similarity = 0.3958
B0BWJ1SP7Q: cosine similarity = 0.3951

User: AEKIGM3H02EOKJ52NKREYSVNJTRA

B0BJY2MYD1: cosine similarity = 0.4546
B09BF8XDF4: cosine similarity = 0.4400
B07DWMCT3X: cosine similarity = 0.4360
B07DWLYGKH: cosine similarity = 0.4294
B000K6CS40: cosine similarity = 0.4195
B07DWY7R2X: cosine similarity = 0.4181
B07DWY44X8: cosine similarity = 0.4171
B07DWM7Y6N: cosine similarity = 0.4150
B07C5DLBZT: cosine similarity = 0.4043
B01LEZLR9M: cosine similarity = 0.3923

User: AHN535MBJTI4BGLTUORFR3TLABLA

B0BR2N43GC: cosine similarity = 0.4791
B09PRKZ6FN: cosine similarity = 0.4541
B003B085BC: cosine similarity = 0.4440
B06XFFMDKK: cosine similarity = 0.4405
B07DJJMXWZ: cosine similarity = 0.4375
B093YCH6LB: cosine similarity = 0.4360
B07Q8F4S4J: cosine similarity = 0.4314
B0002D0CFS: cosine similarity = 0.4270
B00JL5I2AA: cosine similarity = 0.4194
B09BR11LYJ: cosine similarity = 0.4186

User: AH6B5EICC5YHKYSZNBS6NFF3FRHQ

B0928SKWF5: cosine similarity = 0.3561
B08B1LZDDT: cosine similarity = 0.3521
B08GM1RLCT: cosine similarity = 0.3521
B09BYVPFTT: cosine similarity = 0.3500
B083JJD715: cosine similarity = 0.3500
B09BR11LYJ: cosine similarity = 0.3480
B09Q5KGRV3: cosine similarity = 0.3409

B08PZ5FQ3H: cosine similarity = 0.3359
B093YCH6LB: cosine similarity = 0.3329
B07B4ZTD82: cosine similarity = 0.3297

User: AGNQMZSUPRA3BEM44ITFIFXKGPXQ

B07MVLVHTS: cosine similarity = 0.4197
B09DDQ3T57: cosine similarity = 0.3972
B01NBEI47U: cosine similarity = 0.3813
B00NZ80QVW: cosine similarity = 0.3712
B07VS1VTL1: cosine similarity = 0.3361
B01MUYVODO: cosine similarity = 0.3163
B081T636X2: cosine similarity = 0.3134
B083YRNPSR: cosine similarity = 0.3106
B09MHX8NLX: cosine similarity = 0.3056
B07WRRFHKY: cosine similarity = 0.3028

User: AELHFSFNZGMUKRQ6Q2MYN367HZUA

B0002DURLC: cosine similarity = 0.3318
B09M7DR4WM: cosine similarity = 0.3142
B09G5XH5YD: cosine similarity = 0.3129
B007TXT27S: cosine similarity = 0.3026
B00J8STBMI: cosine similarity = 0.2891
B0002E2S9A: cosine similarity = 0.2801
B0002E2XCW: cosine similarity = 0.2772
B0987C2TBZ: cosine similarity = 0.2772
B00IMBYZ5U: cosine similarity = 0.2652
B09M7F7LFB: cosine similarity = 0.2641

User: AE6PSHVBGROWTJMPVBEJZGHNRUQA

B01K7J1H8M: cosine similarity = 0.4923
B01K7J1ECG: cosine similarity = 0.4848
B0002CZVS2: cosine similarity = 0.4551
B0BG64YHQY: cosine similarity = 0.4297
B0064RN6UY: cosine similarity = 0.4194
B09G5HY43T: cosine similarity = 0.4147
B01K7J1G2Y: cosine similarity = 0.4049
B006XY8N9M: cosine similarity = 0.3967
B0B8DR9RGD: cosine similarity = 0.3912
B0BZ1XQX97: cosine similarity = 0.3793

User: AGMYRILPTJDOK7WL50YHYGRYVXMQ

B07JV7SY83: cosine similarity = 0.9560
B07RGK7WBF: cosine similarity = 0.7788
B00DCJ9DCW: cosine similarity = 0.7639
B081VJQ5TX: cosine similarity = 0.7542
B0BWPRCJ32: cosine similarity = 0.7039
B07HM16JZB: cosine similarity = 0.6907
B0188QKTB2: cosine similarity = 0.6877

B0BWVPQTVN: cosine similarity = 0.6733
B0BWVR4S6H: cosine similarity = 0.6668
B07PVKLGTY: cosine similarity = 0.6644

User: AHPIKIUV07ZBNW4K40LRCDRWLQYA

B083BTY7FK: cosine similarity = 0.6083
B083BG1416: cosine similarity = 0.6037
B093WDHWW1: cosine similarity = 0.6030
B09PRKZ6FN: cosine similarity = 0.6009
B0951W3WCF: cosine similarity = 0.5964
B0B176CWVM: cosine similarity = 0.5958
B099NFZP4S: cosine similarity = 0.5947
B07RP68CXC: cosine similarity = 0.5937
B016B3KTUU: cosine similarity = 0.5918
B0769JX3RR: cosine similarity = 0.5910

User: AGQH5PDMQJNYSMTVEYGDH64S2KZA

B0002CZVS2: cosine similarity = 0.3679
B07M81J38F: cosine similarity = 0.3631
B00EL7666K: cosine similarity = 0.3629
B0BJY2MYD1: cosine similarity = 0.3590
B09G5PFGV6: cosine similarity = 0.3574
B00IXXCT9G: cosine similarity = 0.3561
B09G5HY43T: cosine similarity = 0.3545
B0B8DR9RGD: cosine similarity = 0.3539
B0064RTSUQ: cosine similarity = 0.3504
B07DWM7Y6N: cosine similarity = 0.3500

User: AHL67QIBKSTRQMFTJPJK3RJ4KJFQ

B076BDJ965: cosine similarity = 0.4357
B07MDLZ8HC: cosine similarity = 0.4281
B074PR36PQ: cosine similarity = 0.4187
B0B1ZD5SXF: cosine similarity = 0.4148
B01MQ666AX: cosine similarity = 0.4143
B01FM8JIJS: cosine similarity = 0.4140
B01CQKCVDU: cosine similarity = 0.4095
B01N7KFOT2: cosine similarity = 0.4071
B00ZH55DJQ: cosine similarity = 0.4033
B005MOCK9M: cosine similarity = 0.4004

User: AG7AKOBJWYLRDRZ4U4CSDM2L3BVQ

B07BZ7LHRL: cosine similarity = 0.3070
B08KX96FX4: cosine similarity = 0.2873
B079W7NWNH: cosine similarity = 0.2863
B079YDQRSJ: cosine similarity = 0.2821
B0BZ1XQX97: cosine similarity = 0.2794
B07JW4Y9YS: cosine similarity = 0.2792
B09G5KLKX2: cosine similarity = 0.2778

B09G5PFGV6: cosine similarity = 0.2759
B0B8LQFN3C: cosine similarity = 0.2740
B0064RTSUQ: cosine similarity = 0.2733

User: AEYKIWKJUKE2EGISGEQV2XO4IVGA

B00IFHCOUO: cosine similarity = 0.4460
B00IFHCOAE: cosine similarity = 0.4411
B00IFHCOGS: cosine similarity = 0.4141
B0B8P1YW7L: cosine similarity = 0.3572
B0BZ1XQX97: cosine similarity = 0.3538
B00CLFX4EQ: cosine similarity = 0.3495
B07CBT4SYD: cosine similarity = 0.3492
B09DDQ3T57: cosine similarity = 0.3484
B0046A04LU: cosine similarity = 0.3445
B09M7P8SBH: cosine similarity = 0.3416

User: AHSMJCOKS76IY4QASUXIIRQFS3EA

B08GKXLHY1: cosine similarity = 0.3123
B07VJJ3T1N: cosine similarity = 0.3061
B00IFHCOUO: cosine similarity = 0.2911
B089LXZ4KK: cosine similarity = 0.2884
B08DV1LX3J: cosine similarity = 0.2825
B01KINJI20: cosine similarity = 0.2824
B08TX3TRZD: cosine similarity = 0.2782
B06XFFMDKK: cosine similarity = 0.2760
B0BQF951BC: cosine similarity = 0.2759
B00IFHCOGS: cosine similarity = 0.2740

User: AE4QPRXNFNXMTRSF7SZLJ4MEMP2A

B01K7J1H8M: cosine similarity = 0.4947
B01K7J1ECG: cosine similarity = 0.4858
B0B9M4SJ24: cosine similarity = 0.4709
B0BFKKVQZT: cosine similarity = 0.4576
B0B8DR9RGD: cosine similarity = 0.4532
B00UCUEOG4: cosine similarity = 0.4521
B002HFPVCI: cosine similarity = 0.4428
B09XVGFC6F: cosine similarity = 0.4413
B0002CZVS2: cosine similarity = 0.4383
B015QK3GUO: cosine similarity = 0.4372

User: AEWWDVYR4YKLL0ILS55VQISDPGYQ

B009J8DXQW: cosine similarity = 0.6374
B005SUT9UY: cosine similarity = 0.6276
B005SUT9I6: cosine similarity = 0.6242
B0002E2XCW: cosine similarity = 0.5997
B0987C2TBZ: cosine similarity = 0.5997
B0002E2S9A: cosine similarity = 0.5619
B00CGFRJ2Y: cosine similarity = 0.4528

B005SUVHU4: cosine similarity = 0.4060
B07YD56YNP: cosine similarity = 0.3818
B0BCXKYXVN: cosine similarity = 0.3533

User: AHFP3WHRGWB42KPLXR2TN2DPN4HQ

B07VJJ3T1N: cosine similarity = 0.6233
B06XFFMDKK: cosine similarity = 0.6035
B089LXZ4KK: cosine similarity = 0.6021
B0B4BB58CX: cosine similarity = 0.6017
B09ZTWRRPR: cosine similarity = 0.5955
B0BHQ6TF9Q: cosine similarity = 0.5953
B004FRHKA2: cosine similarity = 0.5845
B07XNPSC4K: cosine similarity = 0.5805
B0BQF951BC: cosine similarity = 0.5796
B08PHKFP6G: cosine similarity = 0.5792

User: AH55IBNS320BFM37T2AUX4JR5P6Q

B01K7J1H8M: cosine similarity = 0.5943
B01K7J1ECG: cosine similarity = 0.5872
B0002CZVS2: cosine similarity = 0.5214
B0064RN6UY: cosine similarity = 0.5006
B0B8DR9RGD: cosine similarity = 0.4908
B01K7J1G2Y: cosine similarity = 0.4894
B0BG64YHQY: cosine similarity = 0.4816
B00UCUEOG4: cosine similarity = 0.4744
B006XY8N9M: cosine similarity = 0.4683
B0BFKPM8MQ: cosine similarity = 0.4490

User: AH7BPIYZKJN322GXVLVLJMZ3YPMA

B08R3FDHV9: cosine similarity = 0.5069
B08B1LZDDT: cosine similarity = 0.5041
B08GM1RLCT: cosine similarity = 0.5041
B07B4ZTD82: cosine similarity = 0.4963
B07WS2BBVR: cosine similarity = 0.4905
B08N9ZNV8F: cosine similarity = 0.4880
B083JJD715: cosine similarity = 0.4874
B099NFZP4S: cosine similarity = 0.4873
B09BYVPFTT: cosine similarity = 0.4860
B0C899312H: cosine similarity = 0.4843

User: AGAAUEJ3GP5HNRCP54UMGKPNWVUA

B0928SKWF5: cosine similarity = 0.4004
B00IFHCOAE: cosine similarity = 0.3995
B07VBBHH6Z: cosine similarity = 0.3925
B00JLPHKDA: cosine similarity = 0.3894
B09BYVPFTT: cosine similarity = 0.3891
B08B1LZDDT: cosine similarity = 0.3798
B08GM1RLCT: cosine similarity = 0.3798

B09BR11LYJ: cosine similarity = 0.3793
B07G86NNRB: cosine similarity = 0.3756
B088D2WS29: cosine similarity = 0.3736

User: AE4LGWEQHG3GNC3YEIWKBSZN34Q

B01ANNKMI: cosine similarity = 0.4946
B08PZ5FQ3H: cosine similarity = 0.4819
B004FRHKA2: cosine similarity = 0.4815
B08PF5LBPL: cosine similarity = 0.4812
B007VOGU80: cosine similarity = 0.4797
B01CYAR72M: cosine similarity = 0.4782
B07Y1FSYYB: cosine similarity = 0.4720
B07RXP59L: cosine similarity = 0.4714
B00WHLLDWO: cosine similarity = 0.4712
B07XFPZNMNR: cosine similarity = 0.4699

User: AFZXNEY2426L25LOEOWMYDITOCYQ

B093WDHWW1: cosine similarity = 0.5677
B07MCGRLMT: cosine similarity = 0.5620
B08VJ6ZQFR: cosine similarity = 0.5447
B0769JX3RR: cosine similarity = 0.5434
B07RK6FQ9J: cosine similarity = 0.5340
B07WBZ1SFR: cosine similarity = 0.5305
B08XJP52K8: cosine similarity = 0.5255
B08QRGVTP3: cosine similarity = 0.5226
B08CCQKYD2: cosine similarity = 0.5206
B0936WZT3J: cosine similarity = 0.5150

User: AG64SW2Z4TEZG4IWEGEUFEEZBTRQ

B07DWMCT3X: cosine similarity = 0.4914
B07DWY44X8: cosine similarity = 0.4772
B07DWY7R2X: cosine similarity = 0.4661
B07DWLYGKH: cosine similarity = 0.4621
B000K6CS40: cosine similarity = 0.4377
B07DWKWLW1: cosine similarity = 0.4233
B07BLG25CM: cosine similarity = 0.4228
B01LEZLR9M: cosine similarity = 0.4212
B09JV2KKJP: cosine similarity = 0.4204
B09T7P2RNC: cosine similarity = 0.4074

User: AHQIX3TQQCQRG4VLAekkURPGW7EQ

B0002E2NCM: cosine similarity = 0.3886
B0B2M1GPR7: cosine similarity = 0.3675
B08FMNC8BK: cosine similarity = 0.3622
B07MVLYHTS: cosine similarity = 0.3442
B09DDQ3T57: cosine similarity = 0.3424
B0B8LQFN3C: cosine similarity = 0.3262
B07VJJ3T1N: cosine similarity = 0.3119

B07BSS8VZ: cosine similarity = 0.3104
B0B4BB58CX: cosine similarity = 0.3065
B0B96H2CHT: cosine similarity = 0.3052

User: AH2V5TIVT4EYRHEP2EAWKZFQOHFA

B099NFZP4S: cosine similarity = 0.6020
B07Q8F4S4J: cosine similarity = 0.6005
B09NL4ZR9L: cosine similarity = 0.6001
B093YCH6LB: cosine similarity = 0.5904
B0928SKWF5: cosine similarity = 0.5854
B093WDHWW1: cosine similarity = 0.5802
B096YL3M8Q: cosine similarity = 0.5796
B0BCK6L7S5: cosine similarity = 0.5777
B08JLWKYXP: cosine similarity = 0.5755
B09BYVPFTT: cosine similarity = 0.5734

User: AFIMYZ3UMWWZKR2ZFTOQZ2BDGA6Q

B018K9JZW6: cosine similarity = 0.5465
B0B8LQFN3C: cosine similarity = 0.5451
B07JW4Y9YS: cosine similarity = 0.5154
B07C93Z5XZ: cosine similarity = 0.5104
B084T3Q4J4: cosine similarity = 0.5033
B000B6AGAC: cosine similarity = 0.5008
B0B96H2CHT: cosine similarity = 0.4968
B000P5OUE2: cosine similarity = 0.4956
B0BKRBZ29M: cosine similarity = 0.4956
B0B8LK7MTH: cosine similarity = 0.4908

User: AFJJWM45JXP7ONUSAYQWWEIVDLQ

B07JV7SY83: cosine similarity = 0.5765
B08GFJ37C4: cosine similarity = 0.5748
B09DSGZFGFB: cosine similarity = 0.5677
B073YTN4MJ: cosine similarity = 0.5663
B081G99W8C: cosine similarity = 0.5653
B0776RBZF1: cosine similarity = 0.5602
B07KV9RGJM: cosine similarity = 0.5596
B0915WN7T3: cosine similarity = 0.5587
B099FHKBNZ: cosine similarity = 0.5584
B07Y2S4VLX: cosine similarity = 0.5574

User: AFWPWRAHW5G74XM7P2KS0THNA4JA

B08DV1LX3J: cosine similarity = 0.5272
B01M6COY4T: cosine similarity = 0.5151
B06XFFMDKK: cosine similarity = 0.5144
B018SA2CPE: cosine similarity = 0.5113
B08PP67XCM: cosine similarity = 0.5097
B01M7QOF1E: cosine similarity = 0.5076
B077RGVHRJ: cosine similarity = 0.4998

B0BHL53RKC: cosine similarity = 0.4989
B07HQWXHJW: cosine similarity = 0.4954
B0B4BB58CX: cosine similarity = 0.4930

User: AE2BV2H57ERXAPW7SOAXFLWA2S2Q

B07VS1Y4KL: cosine similarity = 0.5686
B09BR11LYJ: cosine similarity = 0.5635
B07WYNVBQG: cosine similarity = 0.5608
B07VJJ3T1N: cosine similarity = 0.5596
B097392GQF: cosine similarity = 0.5594
B0BTHDNV8Z: cosine similarity = 0.5509
B07ZCV6VZS: cosine similarity = 0.5475
B0BHQ6TF9Q: cosine similarity = 0.5465
B09BYVPFTT: cosine similarity = 0.5458
B0CDRP6W4R: cosine similarity = 0.5444

User: AF5BNNFFASTI2LEXYRWROMIUJWYQ

B0B8DR9RGD: cosine similarity = 0.4541
B09MSCVKP8: cosine similarity = 0.4345
B098WXZ874: cosine similarity = 0.4322
B0B8LK7MTH: cosine similarity = 0.4257
B0B8LQFN3C: cosine similarity = 0.4236
B0B96H2CHT: cosine similarity = 0.4117
B09JV2KKJP: cosine similarity = 0.4090
B09T7P2RNC: cosine similarity = 0.4081
B07BLG25CM: cosine similarity = 0.4062
B0C2QC5FJY: cosine similarity = 0.4018

User: AFXVMXIAMROWJSIZ2IQBTX2GF4LA

B08LVQKMKZ: cosine similarity = 0.4329
B07RK6FQ9J: cosine similarity = 0.4227
B0769JX3RR: cosine similarity = 0.4151
B0BFR5LN7M: cosine similarity = 0.4137
B0C23SC3CZ: cosine similarity = 0.4133
B09FXMQMPD: cosine similarity = 0.4127
B016B3KTUU: cosine similarity = 0.4100
B07XNPSC4K: cosine similarity = 0.4093
B08P4G7S11: cosine similarity = 0.4089
B0BCK6L7S5: cosine similarity = 0.4076

User: AHL4BCH2033HFN7KSBTIJUHCJYDA

B09G5HY43T: cosine similarity = 0.3553
B079W7NWNH: cosine similarity = 0.3540
B0B7C52N2F: cosine similarity = 0.3454
B002UL5W4G: cosine similarity = 0.3367
B079YM2QN3: cosine similarity = 0.3211
B09XSC5SMJ: cosine similarity = 0.3197
B000V1K7FG: cosine similarity = 0.3174

B0BZ1XQX97: cosine similarity = 0.3144
B09G5PFGV6: cosine similarity = 0.3123
B07Y949JY1: cosine similarity = 0.3104

User: AEUART4SSCK2ZI72WHHY4X2SQ2YA

B06XFFMDKK: cosine similarity = 0.5601
B07Y7ST6BF: cosine similarity = 0.5176
B073PXWMJ5: cosine similarity = 0.5097
B0B4BB58CX: cosine similarity = 0.5082
B08DV1LX3J: cosine similarity = 0.5068
B09L4NLRGJ: cosine similarity = 0.5065
B077SZ667X: cosine similarity = 0.5064
B07VJJ3T1N: cosine similarity = 0.5049
B089LXZ4KK: cosine similarity = 0.5046
B081T2PZHW: cosine similarity = 0.5044

User: AHYA024VSRJF47XSA3XNLHHDE4VA

B0C2X15CPH: cosine similarity = 0.3976
B097BG4N5T: cosine similarity = 0.3654
B0002GYX50: cosine similarity = 0.3442
B0002GYX78: cosine similarity = 0.3442
B0C6HZHZG9: cosine similarity = 0.3332
B08DKSR8XS: cosine similarity = 0.3287
B07Q47K8LL: cosine similarity = 0.3274
B0BB9RMGZN: cosine similarity = 0.3199
B00EM5UOE6: cosine similarity = 0.3132
B098WXZ874: cosine similarity = 0.3064

User: AEIW6R03HXEF046UOSW3NLLKAONA

B07BLG25CM: cosine similarity = 0.3818
B09JV2KKJP: cosine similarity = 0.3810
B09T7P2RNC: cosine similarity = 0.3794
B07M81J38F: cosine similarity = 0.3779
B07VJJ3T1N: cosine similarity = 0.3774
B076MNSMDP: cosine similarity = 0.3659
B0B8DR9RGD: cosine similarity = 0.3622
B08DV1LX3J: cosine similarity = 0.3603
B06XFFMDKK: cosine similarity = 0.3601
B01GJRTREC: cosine similarity = 0.3596

User: AEIRBLA7WN4R3DFFWOUBDY6UYWEA

B003B085BC: cosine similarity = 0.4370
B00IFHCOAE: cosine similarity = 0.4130
B0BMZHPKTH: cosine similarity = 0.4049
B09BC1S3JS: cosine similarity = 0.4042
B00IFHCOUO: cosine similarity = 0.3974
B07VBBHH6Z: cosine similarity = 0.3765
B00IFHCOGS: cosine similarity = 0.3706

B07VJJ3T1N: cosine similarity = 0.3665
B0046A04LU: cosine similarity = 0.3612
B01D2QAKJY: cosine similarity = 0.3571

User: AE5M7M2VYMM3WHJIBLCHHEU7U0XQ

B01ANNKMI: cosine similarity = 0.7902
B004FRHKA2: cosine similarity = 0.7768
B007VOGU80: cosine similarity = 0.7759
B005LYIW3W: cosine similarity = 0.7503
B004FEH790: cosine similarity = 0.7165
B07SDRG1G9: cosine similarity = 0.6976
B006XT8EW: cosine similarity = 0.6844
B09NVYW22T: cosine similarity = 0.6732
B07RP68CXC: cosine similarity = 0.6689
B08VN2PFMG: cosine similarity = 0.6638

User: AGK24I67HJ0562FE5RKCGI24T3HA

B08GKXLHY1: cosine similarity = 0.3704
B0BB9RMGZN: cosine similarity = 0.3299
B01K1S6S9W: cosine similarity = 0.3120
B0B3VKD9YG: cosine similarity = 0.2948
B08ML47XPR: cosine similarity = 0.2941
B07ZXWW1FP: cosine similarity = 0.2919
B089LXZ4KK: cosine similarity = 0.2877
B00JJIBUQW: cosine similarity = 0.2850
B09BYVPFTT: cosine similarity = 0.2848
B07P6J4W7D: cosine similarity = 0.2838

User: AFEMET7D7E67XAHX00X74S2QRWVQ

B00IFHCOUO: cosine similarity = 0.3129
B07VJJ3T1N: cosine similarity = 0.3091
B00IFHCOGS: cosine similarity = 0.2935
B0774G3FNJ: cosine similarity = 0.2846
B0BHQ6TF9Q: cosine similarity = 0.2828
B00IFHCOAE: cosine similarity = 0.2800
B08149R167: cosine similarity = 0.2764
B07VS1Y4KL: cosine similarity = 0.2748
B06XFFMDKK: cosine similarity = 0.2712
B07WYNVBQG: cosine similarity = 0.2686

User: AEF2RX7ZF5KKRXYURVRY6AKZGLSQ

B09DDQ3T57: cosine similarity = 0.4259
B07MVLVHTS: cosine similarity = 0.4115
B07B16JL73: cosine similarity = 0.4053
B0B8LQFN3C: cosine similarity = 0.3962
B0B8DR9RGD: cosine similarity = 0.3862
B0BSBVQBVR: cosine similarity = 0.3852
B00162ZROK: cosine similarity = 0.3852

B07V66BWL7: cosine similarity = 0.3794
B0B96H2CHT: cosine similarity = 0.3759
B018K9JZW6: cosine similarity = 0.3755

User: AEYWFJUKDVPF6VNYWN7QB4TS7Z4A

B07VJJ3T1N: cosine similarity = 0.5373
B07VBBHH6Z: cosine similarity = 0.5193
B00JLPHKDA: cosine similarity = 0.5169
B09N35V12J: cosine similarity = 0.5108
B0B4BB58CX: cosine similarity = 0.4980
B0BHQ6TF9Q: cosine similarity = 0.4957
B07KG1KPT3: cosine similarity = 0.4825
B00T275IR2: cosine similarity = 0.4805
B09LS8XMW5: cosine similarity = 0.4798
B0BQF951BC: cosine similarity = 0.4791

User: AGEWXW526KOH5743XNOIKVCMVJQA

B0B8P1YW7L: cosine similarity = 0.5493
B00IFHCOUO: cosine similarity = 0.5473
B00IFHCOAE: cosine similarity = 0.5389
B00IFHCOGS: cosine similarity = 0.5012
B00CLFX4EQ: cosine similarity = 0.4784
B07C1GK87F: cosine similarity = 0.4595
B08JQNQKLL: cosine similarity = 0.4584
B07DWY7R2X: cosine similarity = 0.4497
B00JLPJMP4: cosine similarity = 0.4347
B09BF8XDF4: cosine similarity = 0.4294

User: AH2F6A7VS6CDBSJA24NWKBEWPOA

B093WDHWW1: cosine similarity = 0.7094
B07MCGRLMT: cosine similarity = 0.6920
B08VJ6ZQFR: cosine similarity = 0.6918
B0769JX3RR: cosine similarity = 0.6814
B07RK6FQ9J: cosine similarity = 0.6775
B07WBZ1SFR: cosine similarity = 0.6764
B01G1P5UJS: cosine similarity = 0.6691
B07RYQ1X42: cosine similarity = 0.6677
B01CXPE4EW: cosine similarity = 0.6673
B0BYNM15YB: cosine similarity = 0.6643

User: AEOG5GP4YYOR6T7H4DTLTH5VXEVA

B000LPWHSM: cosine similarity = 0.4701
B084T3Q4J4: cosine similarity = 0.4585
B0B8LK7MTH: cosine similarity = 0.4540
B0B8LQFN3C: cosine similarity = 0.4535
B0B96H2CHT: cosine similarity = 0.4407
B000P50UE2: cosine similarity = 0.4161
B07C93Z5XZ: cosine similarity = 0.4157

B018K9JZW6: cosine similarity = 0.4077
B07JW4Y9YS: cosine similarity = 0.4075
B0BKRBZ29M: cosine similarity = 0.4005

User: AECFGXLEX3SU6D4T6C55X2NWMJVQ

B0064RTSUQ: cosine similarity = 0.3857
B09DDQ3T57: cosine similarity = 0.3784
B09G5PFGV6: cosine similarity = 0.3658
B07MVLVHTS: cosine similarity = 0.3623
B0764R1F7Q: cosine similarity = 0.3606
B00EL7666K: cosine similarity = 0.3592
B0BZ1XQX97: cosine similarity = 0.3579
B0B7C52N2F: cosine similarity = 0.3544
B00AYNJGW6: cosine similarity = 0.3520
B09G5L8DV5: cosine similarity = 0.3473

User: AGTQ4C5U0LPC6RNAX57302TU5S3Q

B07K14LBBC: cosine similarity = 0.3167
B08PFDQ2QS: cosine similarity = 0.2948
B00LPTX5A2: cosine similarity = 0.2882
B008S8ACYA: cosine similarity = 0.2757
B01KINJI20: cosine similarity = 0.2653
B09CZTTDLS: cosine similarity = 0.2629
B08KX96FX4: cosine similarity = 0.2611
B09MWVM7YV: cosine similarity = 0.2545
B01DPCONFM: cosine similarity = 0.2537
B0BQLK5J2S: cosine similarity = 0.2520

User: AEOUOKA2NPLB0SG73KGGJTVQIOK6Q

B0002E1UEO: cosine similarity = 0.4300
B09TBB276B: cosine similarity = 0.4170
B000YF2FGS: cosine similarity = 0.4120
B0B8FZYJJ4: cosine similarity = 0.4116
B000ZLRL6A: cosine similarity = 0.4091
B09T82W1SN: cosine similarity = 0.4088
B072HH4W6C: cosine similarity = 0.4087
B0B29HTRS3: cosine similarity = 0.4075
B09T8KZGTC: cosine similarity = 0.4009
B0B8DHDVJZ: cosine similarity = 0.4005

User: AE2TQUALUTZLMEWBULG5K65EOQSQ

B00IFHCOAE: cosine similarity = 0.4088
B00IFHCOUO: cosine similarity = 0.4040
B0B8DR9RGD: cosine similarity = 0.3940
B00IFHCOGS: cosine similarity = 0.3783
B01K7J1H8M: cosine similarity = 0.3765
B01K7J1ECG: cosine similarity = 0.3759
B07B16JL73: cosine similarity = 0.3701

B0BFKQ9QXD: cosine similarity = 0.3675
B09DDQ3T57: cosine similarity = 0.3673
B0073WGCZA: cosine similarity = 0.3650

User: AFBFEQ63CH3PADWCT52FIYQLG7XA

B0B8LQFN3C: cosine similarity = 0.4165
B09PRKZ6FN: cosine similarity = 0.3977
B07JW4Y9YS: cosine similarity = 0.3893
B0928SKWF5: cosine similarity = 0.3891
B09BYVPFTT: cosine similarity = 0.3881
B0B96H2CHT: cosine similarity = 0.3853
B09ZTWRRPR: cosine similarity = 0.3842
B095WDF9MX: cosine similarity = 0.3805
B093YCH6LB: cosine similarity = 0.3804
B06XFFMDKK: cosine similarity = 0.3778

User: AEW3RXLOEMYULYRIDHAKSVNM5KSA

B07MVLYHTS: cosine similarity = 0.4407
B0BZ1XQX97: cosine similarity = 0.4285
B09DDQ3T57: cosine similarity = 0.4198
B0B7C52N2F: cosine similarity = 0.4186
B09G5PFGV6: cosine similarity = 0.4028
B09M7P8SBH: cosine similarity = 0.3887
B00B1ZONFG: cosine similarity = 0.3868
B09XSC5SMJ: cosine similarity = 0.3859
B09G5HY43T: cosine similarity = 0.3856
B00AYNJGW6: cosine similarity = 0.3830

User: AGAZUH6XYIKXLJHGMSB5WA5ZV2KQ

B07DWMCT3X: cosine similarity = 0.4236
B0064RN6UY: cosine similarity = 0.4009
B098KXQJVY: cosine similarity = 0.3979
B01K7J1H8M: cosine similarity = 0.3930
B000K6CS40: cosine similarity = 0.3896
B0BG64YHQY: cosine similarity = 0.3855
B07DWLYGKH: cosine similarity = 0.3794
B009J8DXQW: cosine similarity = 0.3753
B0BFKQ9QXD: cosine similarity = 0.3741
B01K7J1ECG: cosine similarity = 0.3718

User: AFSC65MF425F0NLVTEYRS5VSUDJQ

B0BR2N43GC: cosine similarity = 0.5305
B0002DOCFS: cosine similarity = 0.5280
B0002GX7XY: cosine similarity = 0.5134
B0064BCRQE: cosine similarity = 0.5110
B0002DOCEO: cosine similarity = 0.5054
B0BYJN7JWQ: cosine similarity = 0.4887
B01M011MR4: cosine similarity = 0.4392

```
BOBMZHPKTH: cosine similarity = 0.4160
B09BC1S3JS: cosine similarity = 0.4158
B09SNBQCM8: cosine similarity = 0.4050
```

User: AFJCZJZFN7DAXW4UNXEYBNGF7YWQ

```
B0002CZVAK: cosine similarity = 0.3102
B0002CZVB4: cosine similarity = 0.2988
B0013NFZHQ: cosine similarity = 0.2988
B00I19ENK0: cosine similarity = 0.2959
B000EMLM06: cosine similarity = 0.2486
B004JOA264: cosine similarity = 0.2401
B088DD5M4V: cosine similarity = 0.2350
B0035JI8J8: cosine similarity = 0.2341
B002GK20AU: cosine similarity = 0.2308
B086PGB73T: cosine similarity = 0.2287
```

0.0.5 6 Week 11 [Mar 10 - Mar 16] Hybrid Recommender System

Create three hybrid recommender systems by combining a collaborative filtering model with a content-based model using the following three strategies: - Parallel combination strategy that re-ranks the items by combining the individual rankings from the two models with some aggregation function such as the sum, average, minimum or maximum

```
[64]: def create_hybrid_parallel(cf_recs, cb_recs, users, k=10, aggregation='sum'):
    """
    Create hybrid recommendations by combining collaborative filtering and
    ↪content-based rankings.

    Parameters:
    cf_recs (dict): Dictionary mapping users to their collaborative filtering
    ↪recommendations
    cb_recs (dict): Dictionary mapping users to their content-based
    ↪recommendations
    users (list): List of users to generate recommendations for
    k (int): Number of recommendations to generate
    aggregation (str): Aggregation function ('sum', 'avg', 'min', 'max')

    Returns:
    dict: Dictionary mapping users to their hybrid recommendations
    """
    hybrid_recs = {}

    for user in users:
        # Get recommendations from both models
        cf_items = cf_recs.get(user, [])
        cb_items = cb_recs.get(user, [])
```

```

# Skip if both recommendation lists are empty
if not cf_items and not cb_items:
    continue

# Create a dictionary to store combined scores
item_scores = {}

# Assign scores based on ranking in collaborative filtering
→ recommendations
for i, item in enumerate(cf_items):
    # Higher score for higher ranked items (inverse of position)
    item_scores[item] = k - i if i < k else 0

# Combine with scores from content-based recommendations
for i, item in enumerate(cb_items):
    score = k - i if i < k else 0
    if item in item_scores:
        if aggregation == 'sum':
            item_scores[item] += score
        elif aggregation == 'avg':
            item_scores[item] = (item_scores[item] + score) / 2
        elif aggregation == 'min':
            item_scores[item] = min(item_scores[item], score)
        elif aggregation == 'max':
            item_scores[item] = max(item_scores[item], score)
    else:
        item_scores[item] = score

# Sort items by combined score in descending order
sorted_items = sorted(item_scores.items(), key=lambda x: x[1],
→ reverse=True)

# Select top-k items
hybrid_recs[user] = [item for item, _ in sorted_items[:k]]

return hybrid_recs

# Test different aggregation methods for the hybrid recommender
aggregation_methods = ['sum']

for method in aggregation_methods:
    # Create hybrid recommendations
    hybrid_recs = create_hybrid_parallel(knn_recs, recs_sample, sample_users,
→ aggregation=method)

# Evaluate the hybrid recommendations

```

```

    parallel_results = evaluate(hybrid_recs, filtered_ground_truth, k=10,
    ↪total_items=len(item_profiles))
    print(f"Hybrid Recommender ({method.capitalize()} Aggregation):",
    ↪parallel_results)

# Compare with individual recommenders
print("\nComparison with individual recommenders:")
print("Content-Based:", results) # Results from previous cell
print("KNN Collaborative Filtering:", knn_metrics) # From previous evaluations

```

Hybrid Recommender (Sum Aggregation): (0.08163265306122448, 0.00816326530612245, 0.014654506491241184, 0.0296728215095562, 0.013925950633755837)

Comparison with individual recommenders:

Content-Based: (0.02040816326530612, 0.0020408163265306124, 0.00040816326530612246, 0.0020408163265306124, 0.013717478318879253)

KNN Collaborative Filtering: (0.10922330097087378, 0.011650485436893206, 0.00841981197641613, 0.03142144398212359, 0.587426326129666)

- Switching strategy that uses the recommendations from the collaborative filtering model for some users and the recommendations from the content-based model for other users chosen by a predefined condition.

```

[65]: def create_hybrid_switching(cf_recs, cb_recs, user_rating_counts, users,
    ↪rating_threshold=None):
    """
    Create hybrid recommendations using a switching strategy based on user
    ↪activity.

    Parameters:
    cf_recs (dict): Dictionary mapping users to their collaborative filtering
    ↪recommendations
    cb_recs (dict): Dictionary mapping users to their content-based
    ↪recommendations
    user_rating_counts (dict): Dictionary mapping users to their number of
    ↪ratings
    users (list): List of users to generate recommendations for
    rating_threshold (int): Threshold for switching between models (if None,
    ↪use median)

    Returns:
    dict: Dictionary mapping users to their hybrid recommendations
    tuple: Statistics about the switching (count of CF users, count of CB users)
    """
    hybrid_recs = {}

    # If no threshold provided, use the median number of ratings

```

```

if rating_threshold is None:
    ratings = [user_rating_counts.get(user, 0) for user in users]
    rating_threshold = np.median(ratings)

cf_user_count = 0
cb_user_count = 0

for user in users:
    # Determine which model to use based on user activity
    rating_count = user_rating_counts.get(user, 0)

    if rating_count >= rating_threshold:
        # Active users get collaborative filtering recommendations
        if user in cf_recs:
            hybrid_recs[user] = cf_recs[user]
            cf_user_count += 1
        else:
            # Less active users get content-based recommendations
            if user in cb_recs:
                hybrid_recs[user] = cb_recs[user]
                cb_user_count += 1

    return hybrid_recs, (cf_user_count, cb_user_count)

# Try different thresholds for switching
thresholds = [5] # None will use median

for threshold in thresholds:
    # Create hybrid recommendations using switching strategy
    hybrid_recs, stats = create_hybrid_switching(
        knn_recs, recs_sample, user_rating_counts, sample_users,
        rating_threshold=threshold
    )

    # Evaluate the hybrid recommendations
    switching_results = evaluate(hybrid_recs, filtered_ground_truth, k=10,
        total_items=len(item_profiles))

    threshold_desc = f"median ({stats[0]+stats[1]//2})" if threshold is None
    else threshold
    print(f"Hybrid Switching Strategy (threshold={threshold_desc}):",
        switching_results)
    print(f"  Users with CF recommendations: {stats[0]}, Users with CB
        recommendations: {stats[1]}")

# Compare with individual recommenders
print("\nComparison with individual recommenders:")

```

```
print("Content-Based:", results)
print("KNN Collaborative Filtering:", knn_metrics)
```

Hybrid Switching Strategy (threshold=5): (0.061224489795918366,
0.006122448979591837, 0.017573696145124718, 0.03231292517006803,
0.007671781187458306)

Users with CF recommendations: 96, Users with CB recommendations: 4

Comparison with individual recommenders:

Content-Based: (0.02040816326530612, 0.0020408163265306124,
0.00040816326530612246, 0.0020408163265306124, 0.013717478318879253)
KNN Collaborative Filtering: (0.10922330097087378, 0.011650485436893206,
0.00841981197641613, 0.03142144398212359, 0.587426326129666)

- Pipelining (sequential) strategy where a level of one model is used as input to the other model.

```
[66]: #####
# Pipeline: Content-Based + KNNWithMeans CF using sample_users with Progress Bar
#####

def build_cb_rating_df(users, items, predict_func, min_rating=1, max_rating=5):
    """
    Build a synthetic user-item rating DataFrame by calling the content-based
    predict_rating function for each (user, item) pair.
    """
    rows = []
    # Use a progress bar for users
    for user_id in tqdm(users, desc="Building CB rating matrix"):
        for item_id in items:
            rating = predict_func(user_id, item_id, min_rating=min_rating,
                                  max_rating=max_rating)
            if rating is not None:
                rows.append((user_id, item_id, rating))
    return pd.DataFrame(rows, columns=['user_id', 'item_id', 'rating'])

# 1) Build a synthetic rating DataFrame from content-based predictions for
# sample_users only
all_items = list(item_profiles.keys())
df_cb = build_cb_rating_df(sample_users, all_items, predict_rating)

# 2) Load into Surprise and train a KNNWithMeans model on the synthetic ratings
# for sample_users
reader = Reader(rating_scale=(1, 5))
data_cb = Dataset.load_from_df(df_cb[['user_id', 'item_id', 'rating']], reader)
trainset_cb = data_cb.build_full_trainset()

knn_pipeline = KNNWithMeans(
    k=20,
```

```

sim_options={'name': 'cosine', 'user_based': True},
verbose=False
)
knn_pipeline.fit(trainset_cb)

# 3) Build the anti_testset from the new trainset (which now only contains
↳sample_users)
anti_testset = trainset_cb.build_anti_testset()

# Generate predictions on the anti_testset
predictions_cb = knn_pipeline.test(anti_testset)

# Convert raw predictions into top-10 lists for each user
cb_recs = {}
for uid, iid, _, est, _ in predictions_cb:
    cb_recs.setdefault(uid, []).append((iid, est))
for uid in cb_recs:
    cb_recs[uid].sort(key=lambda x: x[1], reverse=True)
    cb_recs[uid] = [iid for (iid, _) in cb_recs[uid][:10]]

# 4) Filter the ground truth to only include sample_users
filtered_ground_truth_sample = {u: ground_truth[u] for u in sample_users if u
↳in ground_truth}

# 5) Evaluate with your existing function
pipeline_results = evaluate(cb_recs, filtered_ground_truth_sample, k=10,
↳total_items=len(item_profiles))
print("Pipeline (Content+KNNWithMeans) results for sample_users:")
print("Hit Rate, Precision@10, MAP@10, MRR@10, Coverage =", pipeline_results)

```

Building CB rating matrix: 100%| | 100/100 [1:17:51<00:00, 46.72s/it]

Pipeline (Content+KNNWithMeans) results for sample_users:

Hit Rate, Precision@10, MAP@10, MRR@10, Coverage = (0.0, 0.0, 0.0, 0.0, 0.0)

Use each hybrid model to rank the non-rated items for each user.

Regardless of the chosen option you need to Report Precision@10, MAP@10, MRR@10, the hit rate and coverage using ratings 4 in the test set. Compare the results with the models from previous weeks.

```

[67]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cm

# Collect metrics from all recommender types
systems = {
    "Content-Based": results,

```



```

    "KNN Collaborative": knn_metrics,
    "SVD Collaborative": svd_metrics,
    "TopPop Baseline": top_pop_metrics
}

# Add best hybrid recommenders
# For Parallel hybrid
systems["Parallel Hybrid"] = parallel_results

# For Switching hybrid
systems["Switching Hybrid"] = switching_results

# For Pipeline hybrid, use the specific result with multiplier=2 instead of the
↳ whole dictionary
systems["Pipeline Hybrid"] = pipeline_results

# Create a DataFrame to display the comparison
metrics_df = pd.DataFrame({
    "Recommender": list(systems.keys()),
    "Hit Rate": [systems[rec][0] for rec in systems],
    "Precision@10": [systems[rec][1] for rec in systems],
    "MAP@10": [systems[rec][2] for rec in systems],
    "MRR@10": [systems[rec][3] for rec in systems],
    "Coverage": [systems[rec][4] for rec in systems]
})

# Sort by Hit Rate descending
metrics_df = metrics_df.sort_values("Hit Rate", ascending=False)

# Display the table
print("Recommender System Performance Comparison:")
display(metrics_df)

# Create bar chart for visual comparison of hit rate
plt.figure(figsize=(12, 6))
bars = plt.bar(metrics_df["Recommender"], metrics_df["Hit Rate"],
↳ color="skyblue")
plt.ylabel("Hit Rate")
plt.title("Hit Rate Comparison Across Recommender Systems")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()

# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
        f'{height:.3f}', ha='center', va='bottom')

```

```

plt.show()

# Create a radar chart to compare all metrics
metrics = ["Hit Rate", "Precision@10", "MAP@10", "MRR@10", "Coverage"]
# Normalize metrics for radar chart
normalized_df = metrics_df.copy()
for metric in metrics:
    max_val = normalized_df[metric].max()
    if max_val > 0:
        normalized_df[metric] = normalized_df[metric] / max_val

# Radar chart
colors = cm.rainbow(np.linspace(0, 1, len(systems)))

plt.figure(figsize=(10, 8))
angles = np.linspace(0, 2*np.pi, len(metrics), endpoint=False).tolist()
angles += angles[:1] # Close the loop

ax = plt.subplot(111, polar=True)
ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)

plt.xticks(angles[:-1], metrics)

for i, system in enumerate(normalized_df["Recommender"]):
    values = normalized_df.loc[normalized_df["Recommender"] == system, metrics].\
        values.flatten().tolist()
    values += values[:1] # Close the loop
    ax.plot(angles, values, 'o-', linewidth=2, label=system, color=colors[i])
    ax.fill(angles, values, alpha=0.1, color=colors[i])

plt.legend(loc='upper right', bbox_to_anchor=(0.1, 0.1))
plt.title("Normalized Performance Metrics Across Recommender Systems")
plt.tight_layout()
plt.show()

```

Recommender System Performance Comparison:

	Recommender	Hit Rate	Precision@10	MAP@10	MRR@10	Coverage
3	TopPop Baseline	0.257282	0.032524	0.033864	0.118684	0.019646
1	KNN Collaborative	0.109223	0.011650	0.008420	0.031421	0.587426
2	SVD Collaborative	0.082524	0.008252	0.006104	0.022191	0.117878
4	Parallel Hybrid	0.081633	0.008163	0.014655	0.029673	0.013926
5	Switching Hybrid	0.061224	0.006122	0.017574	0.032313	0.007672
0	Content-Based	0.020408	0.002041	0.000408	0.002041	0.013717
6	Pipeline Hybrid	0.000000	0.000000	0.000000	0.000000	0.000000

